

Mold Godot GDScript

Complete Documentation

Version 0.8.0

Installation, architecture, rendering, performance, and release notes

Contents

Mold Godot	8
Features	10
Requirements	10
Getting started	11
1. Install the addon	11
2. Author shapes in the editor	11
3. Author shapes in code	12
4. Update or draw each frame	12
Shapes and styles	12
Transparent-ordering	13
Custom material/style	13
Rendering game UI with Mold	13
Examples	13
Documentation	14
Community	14
License	14
Third-party software	15
Mold Godot for GDScript	16
Create a runtime you own	16
Shared and automatic runtimes	16
Lines and rounded shapes	17
Polylines	17
GPU polylines	17
Partial shapes and billboards	17
Colors and gradients	18
Dashes	18
Update many shapes at once	18
Immediate drawing	19
Mesh cache and statistics	19
More guides	20
Using Mold in code	21
Create and manage a runtime	21
Create shapes and styles	21
Update and draw each frame	21

Choose the next guide	22
Performance	23
Frustum culling	23
Automatic LOD	23
Batch operation and backend choices	23
Mesh generation	24
Local anti-aliasing for 2D shapes	24
Oklab colorspace blending	24
Transparency ordering	25
Selecting a mode	25
Performance scene	25
Local release identity	26
Architecture	27
Choosing who owns the runtime	27
Creating and updating shapes	27
Drawing shapes in batches	27
How shapes get their appearance	28
Depth, transparency, and stencil	28
Authoring in the editor	28
Language and integration	28
How batching works	29
What can share a batch	29
Frustum culling	29
Automatic LOD	29
Updating many shapes	30
Anti-aliasing and render state	30
Bulk position updates	31
Update positions	31
Choose a backend	31
When updates finish	31
Measure the result	31
Performance test metrics	32
Reading the numbers	32
Reading metrics in your own code	32
Running a benchmark	33

JSON fields	33
Making a useful comparison	33
Blending and render state	34
Choosing a blend mode	34
Dual colors and gradients	34
Differences between renderers	35
Depth and sorting	35
Face culling and stencil	35
Custom materials	36
Getting started	36
A minimal shader	36
Register and draw	37
C#	37
GDScript	37
Use a custom material in the editor	38
Reading instance data	38
Packed values for shape shaders	38
Keeping shapes in the same batch	39
Render-state settings	39
Supporting more shapes	39
GPU-polyline materials	40
Immediate drawing	40
Troubleshooting	40
The shape ignores its color	40
Every 2D shape looks rectangular	40
Shapes double during an LOD transition	40
Registration is rejected	40
Mesh generation and caching	41
What needs a different mesh	41
Lines	41
Rounded shapes	41
Spheres, hemispheres, and capsules	41
Tori and billboards	41
Polylines	42
Managing the cache	42
GPU polylines	43

Retained and authored paths	43
Create and update a stream	43
C#	43
GDScript	44
Bounds, materials, and costs	44
Try the example	45
Rendering game UI with Mold	46
Start with the showcase	46
Set up the camera and coordinates	46
Create the shapes once	46
Update the handles	48
Keep native text aligned	48
Cleanup and world-space UI	48
Rendering backends	49
Choose a renderer	49
Colors and effects	49
Depth and stencil	49
UI and limitations	49
Mold Godot for GDScript	50
Create a runtime you own	50
Shared and automatic runtimes	50
Lines and rounded shapes	51
Polylines	51
GPU polylines	51
Partial shapes and billboards	51
Colors and gradients	52
Dashes	52
Update many shapes at once	52
Immediate drawing	53
Mesh cache and statistics	53
More guides	54
Mold Godot for C#	55
Create a runtime you own	55
Shared and automatic runtimes	55
Lines and rounded shapes	56
Polylines	56

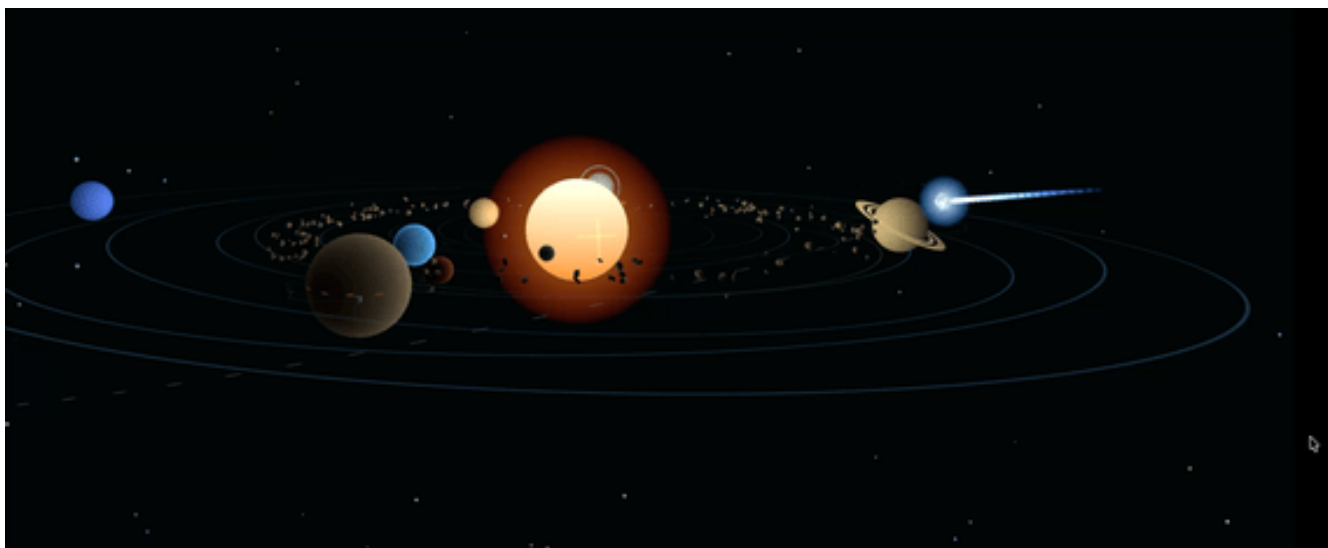
GPU polylines	56
Partial shapes and billboards	57
Colors and gradients	57
Dashes	57
Update many shapes at once	57
Immediate drawing	58
Mesh cache and statistics	58
Managed and SIMD processing	59
More guides	59
Changelog	60
0.8.0 — Local NativeCompatible backport (2026-09-11)	60
Breaking changes	60
Features	60
0.7.1 (2026-09-07)	60
Warning: BREAKING CHANGES	60
Features	60
Documentation	60
0.7.0 (2026-09-07)	60
Features	60
Performance	61
Bug Fixes	61
Documentation	61
0.6.0 (2026-09-06)	61
Warning: BREAKING CHANGES	61
Features	61
Performance	61
Bug Fixes	61
Documentation	62
0.5.1 (2026-09-04)	62
Bug Fixes	62
0.5.0 (2026-09-03)	62
Warning: BREAKING CHANGES	62
Features	62
Improvements	62
Packaging	62
0.4.0 (2026-09-02)	62
Features	62

Bug Fixes	63
Packaging	63
0.3.0 (2026-09-01)	63
Features	63
Mold Godot Proprietary License	64
1. Proprietary rights	64
2. No license granted	64
3. Third-party materials	64
4. Separate agreements	64
5. Disclaimer	64
Third-party notices	65

Mold Godot

[!WARNING] Mold is currently considered experimental until everything is finalized. Breaking changes will happen frequently, so for now it is best to pin local integrations to a specific commit.

A **real-time vector graphics library** for Godot focusing on customizability and performance.



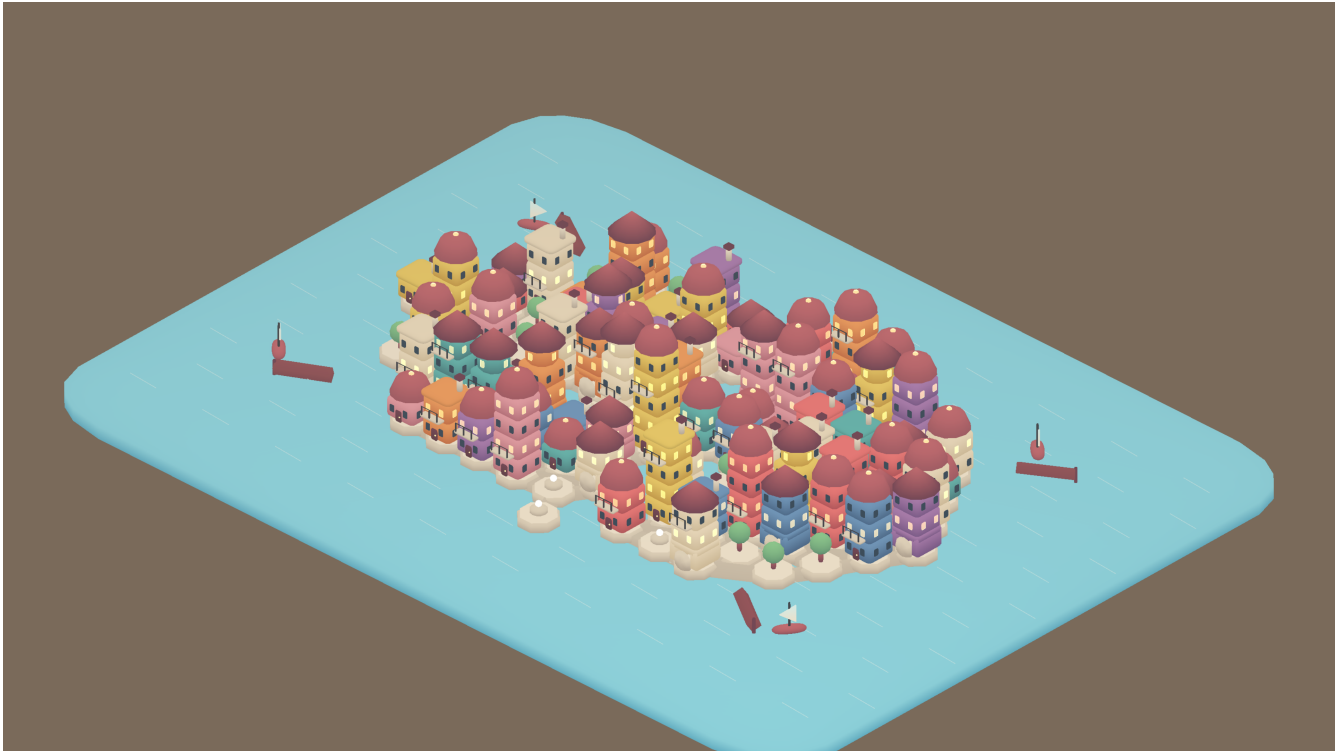
Mold Godot demo

Mold is a real-time vector graphics library for Godot, it is designed with a focus on customizability and high performance.

This library was originally created for the upcoming game [Autopanic](#), but was proven too useful to keep to my own and so a standalone package was created.

The parallel GDScript/C# setup is temporary because code signing on macOS is a hassle. That said, I would still like to move to a GDExtension-based setup in the future to make maintenance easier. Each port directory is also a runnable Godot project, checking out the sample scenes should give you a good idea on how it can be used.

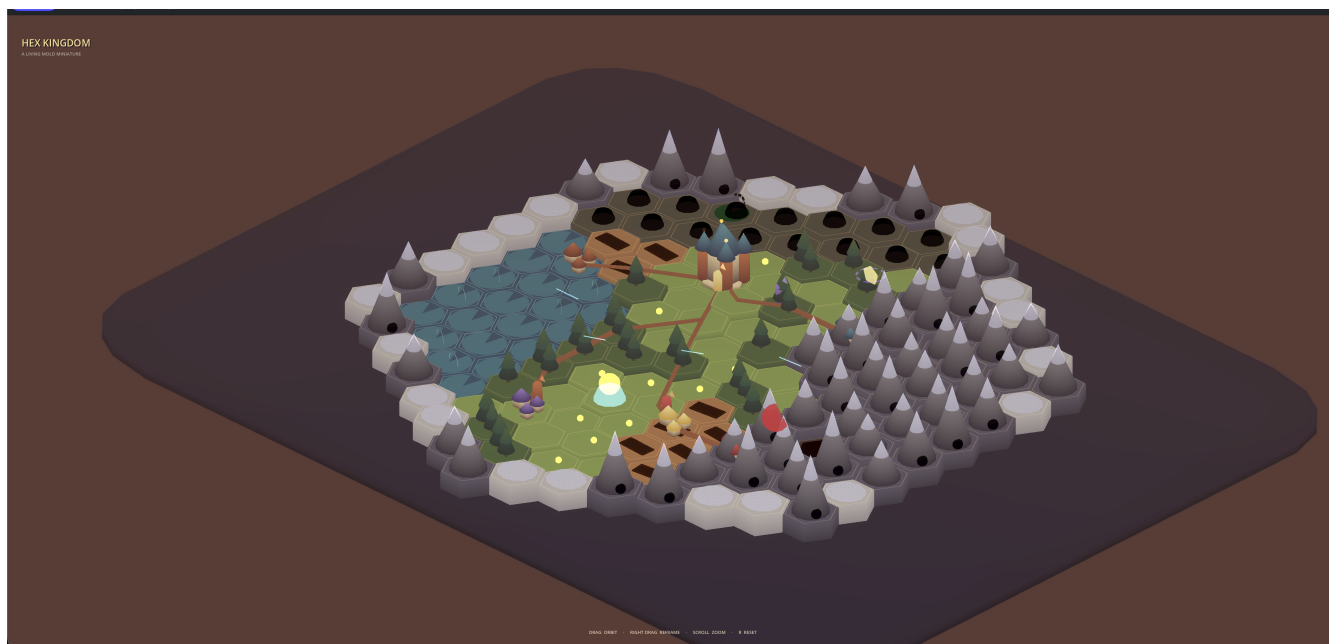
For the biggest showcase so far, check out this [miniature Outer Wilds](#) I made with Mold.



MoldTown



MiniCity

*Mold Godot showcase*

This project is also being developed in parallel with MoldUnity.

Features

- Abundance of 2D/3D geometry primitives. If it's not in here, just tell me the name I'll add it in!
- Capable of creating rounded 3D geometry.
- A monolithic `MoldNode3D` that can draw any shape you wanted, and will be batched into the efficient renderer in runtime.
- Smooth polylines with `MoldPolylineNode3D`, plus GPU streams for animated paths.
- A nice code centric API to create geometry so you don't need to leave your IDE to make asset, and no Node overhead is needed so it can run blazingly fast.
- Utilizes MultiMesh on Godot for efficient rendering.
- Abundance of color blending mode for all 2D/3D geometry primitives, and supports Oklab colorspace blending.
- Automatic dithering cross fade for 3D primitives.
- Designed around retained mode, but immediate mode API is also available with editor preview support.
- Performant by default, with a lot of knobs to twist to suite your need.

Requirements

Requirement	Support
Godot	4.7 or newer; .NET build required for the C# port
.NET	.NET 8 SDK (C# port only)
Renderer	Forward+, Mobile, or Compatibility
Addon dependencies	None

Theoretically this can run on Godot 4.6 but never tested. And 4.6 is required because the support for stencil is crucial to me personally.

The C# runtime and examples require a Godot .NET project. Build the addon assembly before using its global C# classes in scenes.

Due to LDR/HDR framebuffer differences, Compatible renderer is not guaranteed to behavior exactly like the other two.

Getting started

1. Install the addon

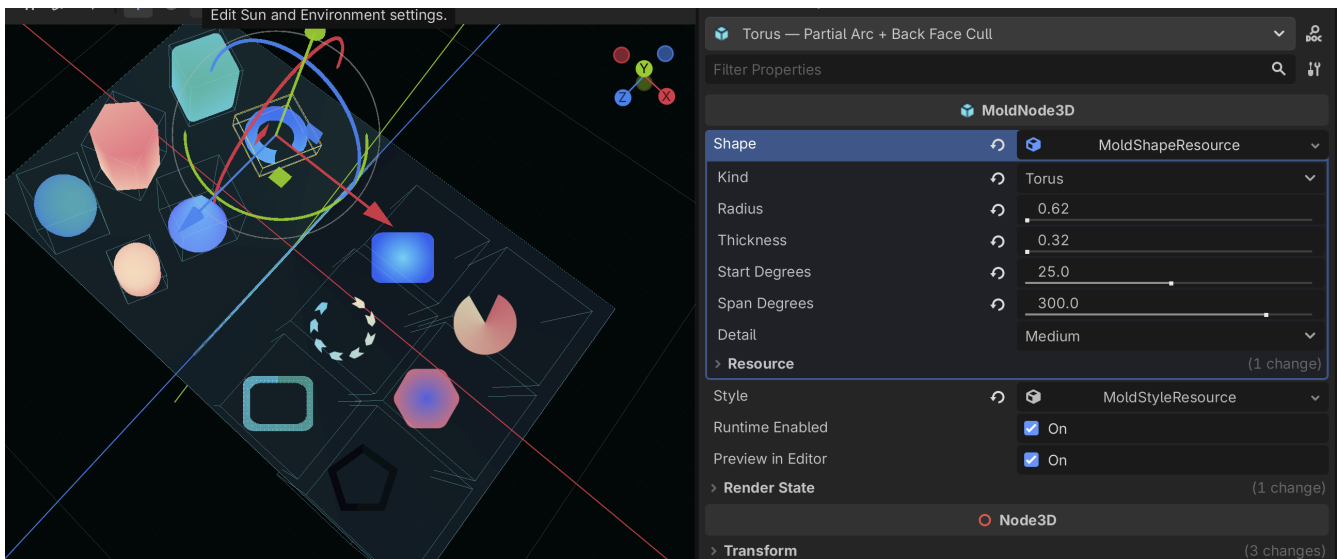
Port	Editor	Web export	API style
GDScript	Standard or .NET Godot	Yes	snake_case
C#	Godot .NET	No	PascalCase

Install only one port in a project. Both release archives install to `addons/mold` and intentionally expose the same global Godot class names.

Copy the selected port's `addons/mold` directory into the consuming project's `addons` directory. When using the C# port, ensure the project includes the copied source files, then build once.

Enable **Mold** under **Project Settings > Plugins** to register the custom selection-bounds gizmo. The plugin does not install an autoload or change project settings, and the runtime API remains usable when the editor plugin is disabled.

2. Author shapes in the editor



Mold Godot node authoring

The simplest way to utilize Mold is by using the `MoldNode3D`

1. Add `MoldNode3D` or `MoldPolylineNode3D` nodes anywhere under the scene or target `SubViewport`.
2. Assign shape or polyline points, style, and optional render state in the Inspector.
3. Use each node's transform for placement and scale.

Both node types automatically join the context owned by their current scene or `SubViewport`. Registration is deferred until scene setup is complete, and a node moved between lifecycle containers releases its old handle before joining the new one. Editor previews update when descriptors or transforms change; runtime rendering still uses retained `MultiMesh` batches.

`MoldPolylineNode3D` exposes a `MoldPolylineResource`; each nested point resource authors local position, tint, and thickness multiplier. Join, cap, closed, miter, and round-subdivision settings live on the polyline resource. Geometry is rebuilt only after an authored resource edit; transforms and visibility update the retained instance without recreating its immutable path.

3. Author shapes in code

Mold is designed to have a very nice code-centric API available, designed to be used for procedural content. The best option is usually to create and keep your own `MoldRuntimeInstance`, this gives you every control you'll want and you'll be responsible for managing its lifecycle.

Alternatively for a tiny script or quick prototype, `MoldRuntime.Create(...)/MoldRuntime.create(...)` is an easier way to lazily create a shared runtime. It stays alive when the current scene changes, and everyone shares the same root-viewport runtime, so remember to release every retained handle. If you need custom settings on the lazily created shared runtime, call `ConfigureShared(...)/configure_shared(...)` before the first shape is created. Again, if you code structure can own the runtime, that is still the version you should reach for.

See each port's README for code examples:

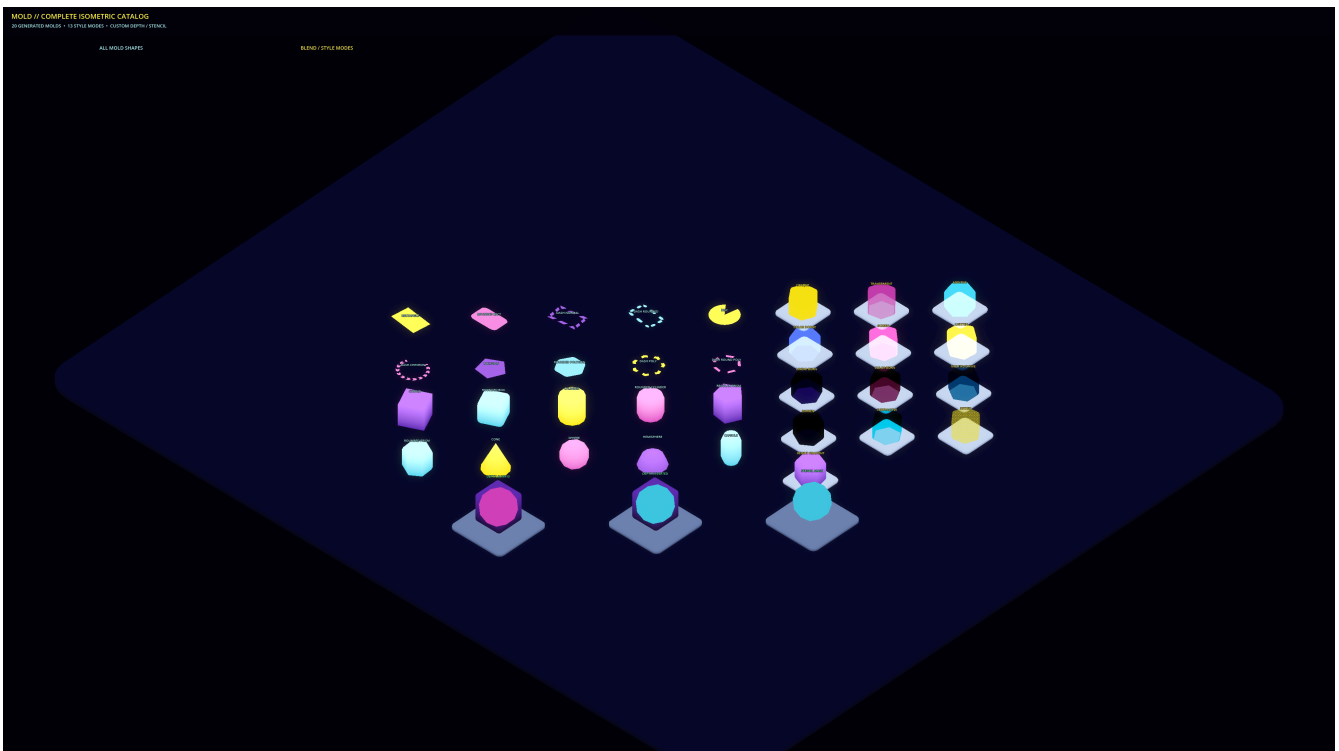
- GDScript guide
- C# guide

4. Update or draw each frame

Keep handles for shapes that persist. Move, recolor, or resize them as values change instead of releasing and recreating them every frame.

For many shapes at once, use the bulk position, color, transform, or line-endpoint APIs. For content you want to describe again each frame, use the immediate API. See [using Mold in code](#) for the workflow and language examples.

Shapes and styles



Mold Godot basic shapes

Mold provides:

- rectangles, rounded rectangles, and their rims;
- discs and rings, including angular spans;
- regular and rounded polygons, including rims;
- cuboids, cylinders, and regular prisms, with rounded variants;

- cones, spheres, capped or uncapped hemispheres, capsules, and partial or full tori.

in which rectangle and cylinder has additional "line" mode which can be used to render them like they are lines.

With the option to be rendered with billboard mode for 2D shapes:

- FaceCamera
- FaceCameraY (only y axis will be roatated, think Doom sprites)

Built-in styles include opaque, transparent, additive, multiplicative, subtractive, linear burn, screen, lighten, darken, color dodge, color burn, dither, and independent single or dual-color evaluation.

`MoldPolyline` draws an open or closed stroke in its local XY plane. It supports per-point color and thickness, miter/bevel/round joins, butt/square/round caps, and solid strokes. Reusing one path object shares its cached mesh and batch; point edits return a replacement path and update only the affected handle. `Auto` uses GPU expansion whenever the backend supports the path and material; explicitly select CPU for cached mesh rendering. See [GPU polylines](#).

Both 2D and 3D primitives can use center-distance radial, bounds radial, or angular dual colors. `DualRadial` uses shape-local distance from the center, including on rim shapes; `DualRadialBounds` preserves a gradient fitted to each 2D shape boundary. On a torus, bounds radial blends from its inner edge to its outer edge; other 3D primitives treat it as `DualRadial`. On 3D primitives, angular color rotates around local Y from +X toward +Z, fitting torus arcs. Both 2D and 3D shapes can use a directional `DualGradient` color fitted to their logical bounds, with a custom axis in world or object space (world up by default). Linear-light RGB and [Oklab](#) interpolation are available independently of the blend mode.

Transparent-ordering

By default Mold uses the so called `NativeCompatible` mode. What this entails is that Mold will internally convert each objects into its individual renderer to ensure consistent transparent ordering behavior with engine transparent objects. This mode is slow by nature, but set as default to prevent confusion.

The additional `Batched` mode provided is the suggested way of using Mold when your project can prevent transparent ordering issue, or can give clear sorting order of each transparent objects.

Check out `performance.tscn` to learn about the performance difference and visual artifact of each mode. More detail on this at [transparent-ordering](#)

Custom material/style

Mold supports registering `ShaderMaterial` with one Mold context to create custom style, and use its world-scoped `MoldMaterialHandle` as a batch identity.

However it is expected that you're highly familiar with shader to utilize it. See the [custom-material guide](#) for more details.

Rendering game UI with Mold

Mold UI is rendered as retained 2D geometry on a camera-aligned plane. Keep text, controls, focus, accessibility, clipping, and automatic layout in Godot's `CanvasLayer/Control` system. For stable HUD rendering, create handles once, mutate them as values change, and use an explicit UI render state with predictable sorting and depth behavior.

See [Rendering game UI with Mold](#) for camera setup, native text alignment, and cleanup.

Examples

The example scenes are the easiest way to learn how Mold works. Open the repository in Godot and run one of these scenes:

- **Basic Molds** — C# · [GDScript](#) the complete shape, style catalog with animation, retained mutations, depth tests, and stencil behavior.
- **Normal Visualization** — C# · [GDScript](#) this is a sample scene to validate the generated mesh's normal is correct.
- **Render State** — C# · [GDScript](#) is a six-panel gallery for less-equal, greater, and always depth tests, depth-write control, stencil write/read masking, and depth-fail stencil writes.

- **Color Gradients** — C# · [GDScript](#) compares radial, angular, and 3D directional dual colors; linear-RGB and Oklab interpolation; and rotating world-space versus object-space gradients.
- **Immediate API Solar System** — C# · [GDScript](#) is a solar system written entirely using immediate API mode. It compares the drawer and pure-immediate submission paths and supports animation scrubbing directly in the editor Inspector.
- **UI Showcase** — C# · [GDScript](#) is a sample scene with the potential ways to create UI using Mold.
- **Node Authoring** — C# · [GDScript](#) shows several primitives generated from the `MoldNode` script.
- **Hex Kingdom** — C# · [GDScript](#) is an imagining of a hex grid strategy game.
- **MiniCity** — C# · [GDScript](#) is an imagining of a MiniMotorway like game.
- **MoldTown** — C# · [GDScript](#) is a small playable builder.
- **Polyline Backend** — C# · [GDScript](#) · an animated wave field with Auto/CPU switching, adjustable line count, and timings.
- **Performance Test** — C# · [GDScript](#) is a scene to test the scalability of Mold.
- **Automatic LOD** — C# · [GDScript](#) is a scene to demonstrate the automatic LOD behavior.
- **Mold Logo** — C# · [GDScript](#) is the scene used to render the Mold logo!
- **Release Showcase** — C# · [GDScript](#) · this is the trailer you saw, rendered directly from here.

Documentation

More detail is available in:

- [Using Mold in code](#)
- [Performance](#)
- [Architecture](#)
- [Batching, culling, and LOD](#)
- [Bulk position updates](#)
- [Performance test metrics](#)
- [Blending and render state](#)
- [Custom materials](#)
- [Mesh generation](#)
- [GPU polylines](#)
- [UI rendering](#)
- [Rendering backends](#)
- [Third-party notices](#)

Community

Join the [Mold Discord server](#) for questions, feedback, and discussion.

Or maybe follow me DK Liao on [X](#) and [Bluesky](#).

Use `#MadeWithMold` to share whatever made with this system I would love to see them!

License

Mold Godot is proprietary software. No rights are granted without a separate written agreement or written permission from DK Liao (Chosen Concept). See the [proprietary license](#) for the complete terms.

Third-party software

Runtime mesh-generation ideas are adapted from MetaMesh. See [Third-party notices](#) for attribution and license terms.

Mold Godot for GDScript

Version 0.8.0 defaults to NativeCompatible transparency. The Performance Test scene switches between Native Compatible and Batched on a 75%-alpha workload. See [Transparency ordering](#) for behavior, costs, and upgrade notes.

This guide covers creating and updating Mold shapes from GDScript. For installation, requirements, and the full list of example scenes, start with the [main README](#). This directory is also a runnable Godot project, so you can open it and try the examples directly.

Create a runtime you own

For most procedural systems, create a `MoldRuntimeInstance` and keep it for as long as that system needs to draw. You control its settings, mesh cache, and cleanup. The following script creates a rounded cuboid and places it one unit above the origin:

```
extends Node3D

var mold: MoldRuntimeInstance
var handles: Array[MoldHandle] = []

func _ready() -> void:
    mold = MoldRuntime.create_instance(self)
    var shape := mold.create(
        MoldShape.cuboid(Vector3(2.0, 2.0, 2.0), 0.4),
        MoldStyle.dual_gradient(Color.CYAN, Color.BLUE),
    )
    shape.set_position(Vector3(0.0, 1.0, 0.0))
    handles.append(shape)

func _exit_tree() -> void:
    for handle in handles:
        if handle.is_valid:
            handle.release()
    mold.dispose()
```

Attach this script to a `Node3D` in a scene with an active `Camera3D` positioned to see the shape. Creating the shape returns a handle; keep that handle to move, recolor, resize, or release it later.

Call `dispose()` when your system is done. It clears the runtime and the shapes it owns. Mold also cleans up if the scene or `SubViewport` goes away, but disposing it yourself makes the lifetime clear.

The later snippets go inside functions on this script and use the same `mold` runtime. Keep any handles you need to update later in `handles` or their own member variables.

Shared and automatic runtimes

For a small script or quick prototype, the shared runtime is shorter:

```
var shape := MoldRuntime.create(
    MoldShape.cuboid(Vector3(2.0, 2.0, 2.0), 0.4),
    MoldStyle.dual_gradient(Color.CYAN, Color.BLUE),
)
```

If you need custom settings, call `MoldRuntime.configure_shared(settings)` before anything uses it. The shared runtime draws in the root viewport and survives scene changes. Every caller shares its capacity, mesh cache, and immediate frame, so remember to release every retained handle you create there.

`MoldRuntime.for_node(self, settings)` is another option. It shares a runtime within the current scene or `SubViewport`. The first call chooses its settings; later calls reuse it and reject conflicting settings. Its handles stop being valid when that scene or viewport leaves the tree.

This automatic runtime is what `MoldNode3D` and `MoldPolylineNode3D` use. For a procedural system that can own its runtime, `MoldRuntime.create_instance(self)` is usually the easier choice to manage.

Lines and rounded shapes

`line_2d` draws a flat rectangle between endpoints, while `line_3d` draws a cylinder. Thickness is the full width of the line. Move the endpoints through the handle when you want to change its length or direction:

```
var handle := mold.create(
    MoldShape.line_3d(Vector3.ZERO, Vector3.UP * 2.0, 0.12, 0.35),
    MoldStyle.opaque(Color.WHITE),
)
handle.set_line_positions(Vector3.ZERO, Vector3.RIGHT * 3.0)
```

Both line types support roundness. For a rounded `line_2d`, pass the billboard mode after thickness, then roundness.

Rectangle lines default to `CORNER_LOCAL` rounding, which keeps the caps round. Use `SHAPE_RELATIVE` if you want the rounding to stretch with the line. The same controls are available on Rectangle resources in the Inspector when line mode is enabled.

Rounded cuboids, cylinders, and prisms keep their edge radius consistent across the shape, with limits based on the shortest dimension. Changing their proportions, including the length of a rounded cylinder line, can select another cached mesh. See [mesh generation](#) for when scaling can save that work.

Polylines

A `MoldPolyline` draws a connected stroke in its local XY plane. Each point can have its own color and thickness multiplier. Paths can be open or closed, with miter, bevel, or round joins and butt, square, or round end caps.

```
var path := MoldPolyline.new([
    MoldPolylinePoint.new(Vector3.LEFT, Color.RED),
    MoldPolylinePoint.new(Vector3.UP, Color.WHITE, 1.5),
    MoldPolylinePoint.new(Vector3.RIGHT, Color.BLUE),
], 0.08, false, MoldPolyline.Join.ROUND, MoldPolyline.Cap.ROUND,
4.0, 8, MoldDash.new(8, 12, 0.25))

var path_handle := mold.create_polyline(
    path, MoldStyle.transparent(Color.WHITE))
path_handle.set_polyline_point(
    1, MoldPolylinePoint.new(Vector3.UP * 1.5, Color.GREEN, 1.5))
```

Reuse the same `MoldPolyline` for unchanged paths so shapes can share its mesh. `set_polyline`, `set_polyline_point`, and `set_polyline_points` replace the handle's path; they do not edit the original path object. Moving the whole handle keeps its mesh. Immediate drawing also supports `draw.polyline(path)`.

To author a path in the editor, add `MoldPolylineNode3D`, create its `MoldPolylineResource`, and edit the nested points in the Inspector. The node provides a preview and handles the runtime registration for you.

GPU polylines

Animated paths can use `MoldGpuPolylineRenderer` to update points without rebuilding a stroke mesh. The polyline backend policy routes Auto paths to GPU expansion whenever it is supported, and otherwise falls back to cached CPU geometry. See [GPU polylines](#) for the API and limitations, and [Polyline Backend](#) for the interactive CPU/GPU comparison demo.

Partial shapes and billboards

A torus can use an angular span to draw part of a ring. A 2D shape can face the camera, or turn only around world Y to stay upright, like a Doom sprite:

```
var torus := mold.create(
    MoldShape.torus(1.0, 0.25, 0.0, PI * 1.5),
    MoldStyle.dual_gradient(Color.VIOLET, Color.CYAN),
)
var badge := mold.create(
    MoldShape.disc(0.5).with_billboard(),
    MoldStyle.transparent(Color.WHITE),
)
var upright_badge := mold.create(
```

```

    MoldShape.disc(0.5).with_billboard(MoldShape.BillboardMode.FACE_CAMERA_Y),
    MoldStyle.transparent(Color.WHITE),
)
var view_line := mold.create(
    MoldShape.line_2d(
        Vector3(-1.0, 0.0, -1.0), Vector3(1.0, 0.0, 1.0), 0.1,
        MoldShape.BillboardMode.FACE_CAMERA,
    ),
    MoldStyle.opaque(Color.WHITE),
)

```

The two badges show the full camera-facing and upright options. The final line keeps its endpoints in 3D space and turns its width toward the camera.

Colors and gradients

`dual_radial()` blends by distance from the local origin of a 2D or 3D shape, including rims. Use `dual_radial_bounds()` to fit the gradient to the shape's boundary:

```

var centered := MoldStyle.dual_radial(Color.WHITE, Color.BLUE)
var fitted := MoldStyle.dual_radial_bounds(Color.WHITE, Color.BLUE)

```

`dual_radial_bounds()` blends from the inner edge to the outer edge on a 3D torus; other 3D primitives treat it as radial. Angular color rotates around local Y from +X toward +Z, fitting a torus's start/span range. Directional gradients also work on 3D shapes. You can choose linear RGB or Oklab interpolation separately from the blend mode. Oklab can give nicer color transitions, but costs extra GPU work. See [blending](#) for the modes and renderer differences.

Dashes

Dashes work on rings, rims, and lines. The count-based pattern below fits 12 periods around the ring, with gaps taking up 45% of each period:

```

var dash := MoldDash.new(
    12,
    12,
    0.45,
    0.0,
    MoldDash.Type.CHEVRON,
    0.8,
)

var ring := mold.create(
    MoldShape.ring(1.5, 0.12, 0.0, TAU, dash),
    MoldStyle.additive(Color.CYAN),
)

var dashed_line := MoldShape.line_3d(
    Vector3.LEFT, Vector3.RIGHT, 0.12, 0.35,
).with_dash(MoldDash.new(10, 10, 0.5))

```

For a line, a count-based pattern stretches across the endpoint span. Fixed-length dash patterns are also available when you want the dash and gap lengths to stay in local-space units.

Flat rectangle lines support normal, chevron, and rounded dash profiles. Cylinder lines use the normal profile. Their gaps are shader cuts, so there are no extra cap faces inside those gaps.

Update many shapes at once

Use the bulk APIs when many shapes change together. Reuse the input arrays across frames to avoid creating more temporary data.

```

var positions := PackedVector3Array()
positions.resize(handles.size())

for index in handles.size():
    positions[index] = calculate_position(index)

mold.update_positions_bulk(handles, positions)

```

`calculate_position(index)` stands for your own animation function. Allocate `positions` once and refill it when updating the shapes. The other bulk methods are:

Change	Method and input arrays
Colors	<code>update_colors_bulk(handles, colors)</code> with a <code>PackedColorArray</code> .
Full transforms	<code>update_transforms_bulk(handles, positions, rotations, scales)</code> with packed position/scale arrays and <code>Array[Quaternion]</code> rotations.
Line endpoints	<code>update_line_positions_bulk(handles, starts, ends)</code> with two <code>PackedVector3Array</code> values.

All arrays in a call must have equal lengths. Mold reads them during the call, skips invalid handles, and uses the last value if you include a handle more than once. The return value counts accepted entries, including duplicates; endpoint updates also skip handles that are not lines. Finish any worker writes before passing the arrays to Mold.

Use position-only updates when rotation and scale are unchanged. Color-only updates likewise avoid touching the transform. See [batching](#) for how these edits reach the renderer.

Immediate drawing

For content you want to describe again every frame, the immediate API saves you from keeping a handle for each shape. `draw()` replaces the previous immediate frame for that context. `with_state()` restores your drawing settings when the callback ends:

```
mold.draw(func(draw: MoldImmediate) -> void:
    draw.with_state(func() -> void:
        draw.style = MoldStyle.opaque(Color.CYAN)
        draw.thickness = 0.04
        draw.dash = MoldDash.new(10, 10, 0.5)
        draw.line(Vector3.LEFT, Vector3.RIGHT)
    )
)
```

Run this drawing code each frame if the content is animated. Commands stay visible until the next immediate frame replaces them or the runtime shuts down. Keep immediate drawing coordinated within a context, since starting another frame replaces what earlier callers drew.

For camera-dependent drawing and editor visualization, extend `MoldImmediateDrawer3D` from a tool script. Mold invokes all drawers in a viewport through one shared frame, so compatible commands still batch together:

```
@tool
extends MoldImmediateDrawer3D

@export_range(0.0, 60.0, 0.1) var preview_time := 0.0:
    set(value):
        preview_time = value
        request_redraw()

func draw_molds(draw: MoldImmediate, camera: Camera3D) -> void:
    draw.style = MoldStyle.opaque(Color.CYAN)
    draw.line(Vector3.ZERO, Vector3.FORWARD * 2.0)
```

Call `request_redraw()` after externally owned preview data changes. Editor cameras automatically trigger another frame when their transform, projection, viewport, or culling mask changes. Editor previews use each shape's requested detail exactly; runtime drawers use the normal screen-space LOD policy. Enable `continuously_redraw` only for editor animation that changes without an Inspector or transform edit.

Mesh cache and statistics

Prewarm shapes before a busy frame if you want to generate their meshes ahead of time. You can also read the context's statistics to see how many shapes and batches it currently holds:

```
mold.prewarm([
    MoldShape.sphere(1.0, MoldShape.Detail.MEDIUM),
    MoldShape.cuboid(Vector3.ONE, 0.35),
])

var stats := mold.statistics
print("%d molds in %d batches" % [stats.active_mold_count, stats.batch_count])
```

`active_mold_count` counts retained shapes. Use `immediate_command_count` and `immediate_batch_count` for immediate drawing. `batch_count` includes both kinds. Call `clear_unused_meshes()` when you want to drop cached meshes that are no longer in use.

Prewarmed meshes still count toward the unused-cache limit. See [mesh generation](#) for the cache settings.

More guides

- [Performance choices](#)
- [Performance test metrics](#)
- [Custom materials](#)
- [Rendering game UI](#)
- [Architecture and runtime ownership](#)

Using Mold in code

For the 0.8.0 default, mode switching, native renderer costs, and custom-material exceptions, see [Transparency ordering](#). Existing batching details below apply to Batched and the unchanged opaque/dither paths.

Create a runtime, keep the handles for shapes that persist, and update them as the game changes. This guide collects the code workflow; start with the [main README](#) for installation and examples.

Create and manage a runtime

For procedural content, create and keep your own `MoldRuntimeInstance`. This gives your system its own settings, capacity, mesh cache, and immediate frame. You are responsible for disposing it when the system is done.

Use `MoldRuntime.CreateInstance(this)` in C# or `MoldRuntime.create_instance(self)` in GDScript. The language guides include complete scripts with creation and cleanup:

- [C# runtime example](#)
- [GDScript runtime example](#)

For a small script or quick prototype, `MoldRuntime.Create(...)/create(...)` uses a shared runtime. It survives scene changes, so release every retained handle. `MoldRuntime.For(...)/for_node(...)` shares a runtime within a scene or `SubViewport`. See [architecture](#) for ownership choices.

Create shapes and styles

Rectangles and cylinders have line modes, available through `MoldShape.Line2D/line_2d` and `MoldShape.Line3D/line_3d`. Thickness is the full line width. Use `SetLinePositions/set_line_positions` to change local endpoints, or the handle's transform to move the whole line.

2D shapes can face the camera with `WithBillboard/with_billboard`. `FaceCamera` turns toward the camera; `FaceCameraY` rotates only around world Y. Rounded 3D shapes fit their edges to their proportions, so changing proportions can require a different cached mesh. `SetScale/set_scale` stretches the existing shape when that appearance is acceptable.

The language guides cover partial shapes, polylines, colors, gradients, and dashes:

- [C# shapes and styles](#)
- [GDScript shapes and styles](#)

See [blending](#) for blend modes and color interpolation, and [mesh generation](#) for mesh reuse.

Update and draw each frame

Keep handles for shapes that persist. Move, recolor, or resize them as values change instead of releasing and recreating them every frame.

For many shapes at once, use the bulk position, color, transform, or line-endpoint APIs. These methods read matching arrays during the call; C# also accepts spans. Call Mold APIs on the main thread.

For content you want to describe again each frame, use the immediate API. Starting another immediate frame replaces the previous one for that context, so keep those calls coordinated. Immediate shapes do not use retained handles.

`MoldImmediateDrawer3D` supports camera-driven drawing and editor previews. The language guides include complete examples:

- [C# immediate drawing](#)
- [GDScript immediate drawing](#)

Choose the next guide

- [Architecture and runtime ownership](#)
- [Batching, culling, and LOD](#)
- [Bulk position updates](#)
- [GPU polylines](#)
- [Custom materials](#)
- [Mesh generation and caching](#)

Performance

For the 0.8.0 default, mode switching, native renderer costs, and custom-material exceptions, see [Transparency ordering](#). Existing batching details below apply to Batched and the unchanged opaque/dither paths.

Performance depends on the workload and transparency ordering mode. NativeCompatible uses individual engine objects; Batched reduces submission overhead when batch-slot ordering is acceptable. This is a guide on every performance implications for Mold. Note that **Always benchmark, especially in build**, before committing to a specific choice.

Frustum culling

Frustum Culling can lower GPU usage by not rendering shapes when they are out of view. However maintaining the culling itself has additional performance cost, which means sometimes it might be just cheaper to draw them at all times.

Godot has two choices:

- None: Draw all things at all times, best if most of the shapes are mostly on screen.
- BatchBounds: Has CPU cost. Calculates the bound of each MultiMesh and submit to Godot for Godot to reject them.

Automatic LOD

Automatic LOD is a feature that allows smooth transition of mesh details. This in itself is a cost saving measure for keeping everything smooth while not needing to show the highest quality of everything at all times.

However Automatic LOD itself comes with three caveats:

- The determination of which LOD to use itself has CPU cost.
- This forces the generation of all mesh detail by default, which will increase memory footprint.
- LOD transitioning shapes will consume two batches of rendering because they're using different mesh, which don't always has performance implication, but something to keep in mind.

If your project has minimal camera distance changes for example an isometric game, turning it off and hand author each mesh detail might be the better choice.

Batch operation and backend choices

Batch operations are available through `MoldContext` for lowering the CPU cost. Currently there are:

- `UpdatePositions(handles, positions)`: position only
- `UpdateTransforms(handles, positions, rotations, scales)`: full transform
- `UpdateColors(handles, colors)`: color only
- `UpdateLinePositions(handles, starts, ends)`: line specific

If you're updating a lot of shapes at once, using these APIs will have significant performance improvements.

Additionally, in C# the batch operation has two backend options available:

- Managed: well, the normal-ish way of updating
- SIMD: utilizing `System.Numerics` for vectorized performance improvements.

SIMD is the default option which generally should run faster, but you can still use the Managed backend if you need it, for example if the SIMD path runs slower due to incompatible hardware which may or may not lead to worse performance.

Mesh generation

By default Mold caches the mesh generated for each shapes, they will reused on every matching ratios. Mesh generation cache is a trade-off between CPU cost and memory footprint and when needed `MoldContext` can use `ClearUnusedMeshes()` to clean up the mesh cache.

Therefore a good practice is to "SetScale" instead of "SetShape" whenever you can get away with it, albiet at the cost of potential non-uniform appearance.

Note that 2D shapes, spheres, cuboids, and hemisphere does not subject to this characteristic due to their uniform mesh. Meaning `SetScale` and `SetShape` for scaling yields the same result.

Additionally the amount of unique mesh used is also proportional to the batches needed render the shapes. This doesn't always translate to performance impact but it is something to keep in mind and avoid if you can get away with it.

Local anti-aliasing for 2D shapes

Mold uses derivative to enable the smooth look of the 2D shapes. This has GPU performance cost, while small, adds up if you're running a very shape heavy game.

There are three options available for the anti-aliasing:

- `High`: uses the Euclidean derivative length, `length(vec2(dFdx(value), dFdy(value)))`, for the best diagonal coverage.
- `Medium`: uses the cheaper `fwidth(value)` approximation.
- `Off`

The default is `High`.

Oklab colorspace blending

Oklab colorspace blending loops great, but it needs to perform additional colorspace conversion both in and out of Oklab colorspace which means extra GPU performance cost.

This is the reason why Oklab is not the default color blending option, please use it conciously and sparingly.

Transparency ordering

Mold defaults to **NativeCompatible**, which gives supported transparent Mold objects individual engine renderers so the engine can sort them alongside its ordinary transparent objects. **Batched** uses batch-slot ordering to reduce submission overhead. Opaque and dither geometry retain their existing rendering paths.

NativeCompatible matches the engine's object-level sorting behavior.

Selecting a mode

```
var mold = MoldRuntime.CreateInstance(this, new MoldWorldSettings
{
    TransparentOrderingMode = MoldTransparentOrderingMode.NativeCompatible,
});
mold.TransparentOrderingMode = MoldTransparentOrderingMode.Batched;

var settings := MoldWorldSettings.new()
settings.transparent_ordering_mode = MoldWorldSettings.TransparentOrderingMode.NATIVE_COMPATIBLE
var mold := MoldRuntime.create_instance(self, settings)
mold.transparent_ordering_mode = MoldWorldSettings.TransparentOrderingMode.BATCHED
```

Native batches use pooled MeshInstance3D nodes, sharing meshes, shaders, and the existing seven-textel appearance data. Each native slot owns a pooled material carrying its index; this avoids Godot's finite global instance-uniform buffer, including the much smaller Compatibility limit. Native material counts therefore grow with native slots. GPU polylines already use MeshInstance3D. Custom-material batches remain batched and are not promised native ordering. Both ports use AABB-center sorting for native shapes.

C# settings resources: The resource class now resides in `MoldWorldSettings.cs` so Godot can save and reload it. Its public type and properties are unchanged. Settings resources that reference the old `MoldTypes.cs` script must be recreated or retargeted to the new settings script.

C# migration: Batched no longer sorts alpha-transparent instance uploads back-to-front. It preserves slot order, matching the GDScript port and current Mold behavior. Selecting Batched is therefore not an exact 0.7.1 visual-compatibility switch. Transparent LOD restrictions are preserved.

`NativeCompatible = 0` and `Batched = 1` are stable enum values. New and omitted settings default to native mode; invalid values normalize to native mode. Changing modes preserves handles and switches the rendering path. Native pools are created on first use and reused until their owning batch/world is retired.

Performance scene

The Performance Test scene offers **Native Compatible** and **Batched** under **Transparency ordering**. Its four gradient styles use alpha 0.75 and back-face culling. The existing 20,000-object default, count presets, animation, update-mode controls, and Manual LOD are retained. Unity also retains its Retained/Immediate control.

Switching mode keeps the workload and other controls, discards pending measurements, and shows “Warming up...” for 120 completed frames before collecting a fresh 120-frame rolling window. Count/API rebuilds preserve selection. Selecting the already-active mode does nothing.

For automatic comparisons, pass `--benchmark-ordering=native` or `--benchmark-ordering=batched`; omission selects native. Unsupported values exit with an error before creating the world. Godot user arguments follow its `-- separator`. Existing `--benchmark-count`, `--benchmark-warmup`, and `--benchmark-frames` options remain available. Results identify `transparent_ordering_mode`, `workload_alpha`, and the rendering backend.

Native mode delegates individual renderer visibility to the engine. Mold's GPU-culling statistics and logical batch count do not represent actual native draw calls. The HUD measures CPU and wall-clock frame time, not GPU execution time. Use the engine profiler for actual GPU/draw counts. Large native workloads can be substantially slower; choose Batched explicitly for maximum throughput when its ordering is acceptable.

Local release identity

This is a focused backport onto the supplied 0.7.1 packages, delivered locally as 0.8.0. It is distinct from the broader existing repository tag named v0.8.0. These packages do not add OIT, polygons, curves, or later geometry-deformation features. Consult the included files and local validation report rather than assuming the public v0.8.0 tag describes this build.

Architecture

Mold keeps your shapes, meshes, and rendering batches in a `MoldWorld`. You talk to it through `MoldContext`, so you don't need to create or manage a renderer yourself. This guide explains what happens behind that API.

Choosing who owns the runtime

For most procedural systems, create a `MoldRuntimeInstance` with `MoldRuntime.CreateInstance(node)` in C# or `MoldRuntime.create_instance(node)` in GDScript. Keep it for as long as the system needs to draw, then dispose it. This gives you control over its settings, mesh cache, and cleanup.

There are also two shortcuts:

- `MoldRuntime.For(node)/MoldRuntime.for_node(node)` shares a runtime within the current scene or `SubViewport`. This is what the authoring nodes use.
- `MoldRuntime.Create(...)/MoldRuntime.create(...)` uses the shared runtime for the root viewport. It is convenient for a small script, but it survives scene changes, so remember to release the shapes you create.

Mold hides its renderer nodes in the scene tree. Owned and automatic runtimes also shut down when their scene or `SubViewport` goes away. Disposing your owned runtime yourself is still the clearest way to say that your system is done.

Creating and updating shapes

Creating a shape returns a `MoldHandle`. Keep that handle and use it to move, recolor, resize, or release the shape. Mold stores its own copy of the shape and style, so changing the original descriptor later does not change an existing shape.

Handles also remember which world and slot they belong to. When a slot is reused, it gets a new generation number. This prevents an old handle from accidentally editing a newer shape, and keeping a shape handle does not keep its world alive.

Mold tracks transform changes separately from color and other property changes. Moving a shape therefore does not require repacking its unchanged colors, dashes, or emission. Releasing a shape fills its batch slot with the last entry, so batches stay compact without shifting every remaining shape.

Rectangle and cylinder lines store their endpoints in local space. Mold combines the line's placement with the handle's transform. Moving the endpoints usually reuses the same mesh; changing the proportions of a rounded cylinder line can require a different rounded mesh.

Drawing shapes in batches

Mold groups shapes that can render together into batches. Each batch owns a `MultiMeshInstance3D`, a `MultiMesh`, and a material. Shapes need compatible meshes, blend modes, render states, and custom materials to share a batch. Colors, gradients, dashes, and transforms can differ within that batch. See [batching](#) for the settings that affect this.

Transforms are sent through Godot's bulk `MultiMesh` buffer, so Mold can update a whole batch without making a separate engine call for every shape.

The other shape properties live in a floating-point texture. The shader reads the entry belonging to each shape. This is how differently colored or dashed shapes can share the same material. The [custom-material guide](#) lists the layout if you need to read it in your own shader.

Generated meshes are cached and can be shared by several batches. Built-in shaders are also reused when their blend, depth, stencil, and face-culling settings match.

How shapes get their appearance

The six 2D shape families all start with the same quad. The fragment shader calculates which pixels belong to the shape using a signed-distance function (SDF). It also handles rims, dashes, and smooth edges. `High` local anti-aliasing is the default; `Medium` and `Off` reduce the shader work.

3D shapes use meshes generated at runtime. Some shapes also change their vertices in the shader. For example, capsules can change height while keeping their caps round, without generating a separate mesh for every height. CPU polylines have their own generated stroke meshes. GPU polylines instead expand fixed carrier meshes from point and segment textures in a spatial shader. Their streams belong to the world and are cleaned up with it. See [GPU polylines](#) for backend selection and streaming updates. See [mesh generation](#) for the cache and shape details.

Mold starts by converting the colors you supply from sRGB to linear RGB. Oklab gradients take an extra conversion through Oklab for the blend. Mold converts the final output back to sRGB when the renderer needs it. This keeps ordinary colors similar across renderers, although HDR emission and advanced blending can still look different in Compatibility. See [blending](#) for those differences.

Depth, transparency, and stencil

Mold renders into the scene viewport, using Godot's cameras, visual layers, and depth buffer. `MoldRenderState` controls how a batch interacts with them. Godot exposes three depth-test choices through this rendering path: `LessEqual`, `Greater`, and `Always`. Stencil reads use the transparent pass, even for a shape whose style is otherwise opaque.

The C# renderer sorts ordinary `Transparent` shapes from back to front within each batch when depth writes are disabled. Those shapes use discrete automatic LOD changes, even in `Continuous` mode, because a crossfade would split each shape across two separately sorted draws. Explicitly enabling depth writes allows the continuous transition again.

The GDScript renderer currently has neither that per-instance transparent sort nor the automatic fallback to discrete LOD. Keep this difference in mind when working with overlapping transparent shapes. Custom shaders also need to handle their own transparency and LOD behavior.

Authoring in the editor

`MoldNode3D` and `MoldPolylineNode3D` let you author retained shapes in the editor. Each node registers a handle with the automatic runtime for its scene or `SubViewport`, then updates that handle when you edit its resources or transform. It releases the handle when it leaves, including before joining a different scene or viewport runtime.

Editor previews use cached meshes and the same shader logic as runtime rendering. The preview child is internal and is not saved into your scene.

Language and integration

The C# port targets .NET 8 and uses Godot's `Vector2`, `Vector3`, `Quaternion`, `Color`, and `Transform3D` types. Most of the API lives in `Mold`. The `Radius` and `Diameter` helpers live in `Mold.Math`, so you can import them only when you want those names in your code.

How batching works

For the 0.8.0 default, mode switching, native renderer costs, and custom-material exceptions, see [Transparency ordering](#). Existing batching details below apply to Batched and the unchanged opaque/dither paths.

Batching lets Mold draw many shapes together using Godot's `Multimesh`. You don't need to arrange the batches yourself, but the choices you make about meshes and render states affect how many batches Mold needs.

What can share a batch

Shapes can share a batch when they use the same mesh, blend mode, custom-material handle, render state, and local anti-aliasing quality. Render state includes visual layers, sorting order, depth testing and writing, stencil, and face culling.

Transforms, colors, gradients, dashes, and emission can differ without splitting that batch. Large retained workloads are also divided into groups of slots, so matching shapes are not guaranteed to fit in one batch. Changing a setting that requires a different batch moves only the affected shape. If you change it back, Mold can group it with matching shapes again.

Size changes often reuse the same mesh, but there are exceptions. Rounded cuboids, cylinders, and prisms use meshes fitted to their proportions, so changing those proportions can need another mesh. Detail level, prism side count, hemisphere caps, and whether a torus is partial or full also affect mesh sharing. See [mesh generation](#) for more details.

Frustum culling

By default, Mold calculates a bounding box around each batch and gives it to Godot. Godot can skip the batch when that box is outside the camera's view. This is batch-level culling: a batch that is partly visible can still draw shapes that are off screen.

Set `Culling` to `None` in `C#`, or `culling_mode` to `MoldWorldSettings.CullingMode.NONE` in GDScript, if you want to skip Mold's bounds calculation. If your shader moves vertices beyond their normal bounds, add `CullingBoundsPadding/culling_bounds_padding` to leave enough room.

Calculating bounds has a CPU cost. If most shapes are always on screen, benchmark both options before choosing. The [performance guide](#) goes through this trade-off.

Automatic LOD

LOD lowers mesh detail when a shape takes up less screen space. Mold uses the viewport's active `Camera3D` and never chooses a detail level above the one you authored.

There are three modes:

- `Continuous`, the default, fades between two mesh detail levels using dithering. Both meshes are drawn during the transition.
- `Automatic` switches between levels directly. A small threshold margin (hysteresis) stops shapes from switching back and forth near a boundary.
- `Manual` keeps the detail you authored. This is a useful choice for a fixed camera or UI where you already know how much detail you need.

2D SDF shapes, polylines, unrounded cuboids, and unrounded regular prisms keep fixed meshes. Automatic LOD does not simplify them.

`LodThresholdPixels` sets the screen-size boundaries, and `LodBias` shifts the quality choice.

`LodEvaluationBudget` limits how many retained entries Mold checks in a frame, taking turns through the list. With an orthographic camera, Mold can skip repeat checks until a relevant camera, viewport, or shape change.

These examples show the default quality settings:

```
var settings = new MoldWorldSettings
{
    LocalAaQuality = MoldLocalAaQuality.High,
    LodMode = MoldLodMode.Continuous,
    LodThresholdPixels = new Vector4(16, 32, 64, 128),
    LodEvaluationBudget = 4096,
};

var settings := MoldWorldSettings.new()
settings.local_aa_quality = MoldWorldSettings.LocalAaQuality.HIGH
settings.lod_mode = MoldWorldSettings.LodMode.CONTINUOUS
settings.lod_threshold_pixels = Vector4(16.0, 32.0, 64.0, 128.0)
settings.lod_evaluation_budget = 4096
```

Use `MoldLodMode.Automatic` or `MoldLodMode.Manual` in C#, and `MoldWorldSettings.LodMode.AUTOMATIC` or `.MANUAL` in GDScript, to change modes. Statistics include counts by detail level, active transitions, and how many shapes changed LOD batches in the latest frame.

C# makes one exception for ordinary Transparent shapes with depth writes disabled: they switch detail directly so a two-mesh fade does not break their back-to-front ordering. GDScript currently keeps the two-mesh transition and does not sort instances within a transparent batch.

Updating many shapes

Use the bulk APIs when you are changing a lot of shapes at once:

Change	C#	GDScript
Positions	<code>UpdatePositions()</code>	<code>update_positions_bulk()</code>
Full transforms	<code>UpdateTransforms()</code>	<code>update_transforms_bulk()</code>
Colors	<code>UpdateColors()</code>	<code>update_colors_bulk()</code>
Line endpoints	<code>UpdateLinePositions()</code>	<code>update_line_positions_bulk()</code>

Pass matching arrays, or spans in C#, for handles and values. Mold reads them during the call, skips invalid handles, and uses the last value if you include a handle more than once.

See [bulk position updates](#) for backend selection and update examples.

Mold tracks changed transforms separately from changed properties. Position-only updates reuse the existing rotation, scale, and shape bounds where possible. Only the affected groups need their bounds recalculated. Bounds can shrink after a move or removal, as well as grow.

C# defaults to `MoldPositionUpdateMode.SIMD` for its bounds work. It uses `System.Numerics.Vector4` when hardware acceleration is available and the group is large enough; otherwise it uses the scalar calculation. You can select `Managed` if you want to compare the two on your target hardware.

Unchanged batches usually skip synchronization. When an upload is needed, Godot still receives a complete transform buffer or property texture for the batch's allocated capacity. Updating fewer CPU records does not mean Mold sends only those bytes to the GPU. Camera changes and C# transparent sorting can also require another upload.

Anti-aliasing and render state

Local anti-aliasing quality is a world setting. `High` gives the smoothest diagonal edges, `Medium` uses a cheaper approximation, and `Off` leaves hard edges. Set `LocalAaQuality/local_aa_quality` before creating the runtime.

Face culling is disabled by default. Use `MoldFaceCull.Back` in C# or `MoldRenderState.FaceCull.BACK` in GDScript when you want to hide back faces. Depth and stencil settings are covered in [blending](#).

Bulk position updates

Use bulk updates when many shapes move together. Pass the handles and their new positions in matching arrays, then keep those arrays for the next update. This avoids a separate setter call for every shape.

Update positions

```
int updated = context.UpdatePositions(handles, positions);

var updated := context.update_positions_bulk(handles, positions)
```

Use matching handle and position arrays, or spans in C#. Positions use Godot's `Vector3`. The call reads your inputs and finishes the update before returning.

Mold skips stale handles and handles from another world. If a handle appears more than once, the last position wins. Accepted counts can include positions that did not change. The input lengths must match.

Position-only updates keep the existing rotation, scale, and shape. Use the full-transform, color, or line-endpoint APIs for those changes; see [batching](#).

Choose a backend

C# has two choices:

- Managed uses the scalar calculation.
- SIMD, the default, uses `System.Numerics.Vector4` to calculate several bounds values together when hardware acceleration is available.

Choose the initial mode in `MoldWorldSettings.PositionUpdateMode`. Settings are copied when the runtime is created, so use the context setter for later changes:

```
context.SetPositionUpdateMode(MoldPositionUpdateMode.Managed);
var requested = context.PositionUpdateMode;
var effective = context.EffectivePositionUpdateMode;
bool available = MoldContext.IsPositionUpdateModeAvailable(
    MoldPositionUpdateMode.SIMD);
```

Each runtime has its own choice. Unsupported hardware falls back to Managed while preserving the requested SIMD mode. Small groups use the scalar calculation in either mode. GDScript uses its own synchronous implementation and has no backend selector.

When updates finish

Both C# backends and the GDScript API finish before returning. SIMD speeds up the bounds calculation; it does not move the work to worker threads.

Call Mold APIs on the main thread. If other work calculates your input positions, finish it before passing the results to Mold. You can reuse the input arrays as soon as the call returns.

Measure the result

Compare full frame time in a build on your target hardware. Faster bounds calculations do not necessarily mean a proportionally faster frame: Godot still receives a complete transform buffer or property texture when a batch needs an upload.

The C# performance scene has a Managed/SIMD selector. Benchmarks accept `--benchmark-mode=managed` or `--benchmark-mode=simd` after Godot's `--` separator. See [performance metrics](#) for the counters and benchmark arguments.

Performance test metrics

For the 0.8.0 default, mode switching, native renderer costs, and custom-material exceptions, see [Transparency ordering](#). Existing batching details below apply to Batched and the unchanged opaque/dither paths.

The performance scene helps you see where time is going as you increase the number of shapes. Start with frame time, then use the CPU timings to see which part of the work gets more expensive. **Always compare in a build on your target hardware** before committing to a performance choice.

Open the [C# scene](#) or the [GDScript scene](#) to try it.

Reading the numbers

The HUD shows the last 120 frames and refreshes twice per second. Changing the shape count or pausing/resuming animation clears that window. Give the numbers a moment to settle after a change, since the first frames can include mesh creation and initial uploads.

Metric	What it tells you
Frame mean / p95	Average frame time and the time that about 95% of frames fall below. A high p95 can reveal slow frames hidden by the average.
Simulation	Time spent calculating the animated positions in the sample.
Bulk update	Time spent passing those positions to Mold, including validation and any transform or bounds work done by that call.
Renderer CPU	Time Mold spends preparing and submitting rendering, including LOD, bounds, packing, and uploads.
LOD CPU	The part of Renderer CPU spent checking mesh detail. Manual LOD still has a small check cost.
Upload CPU	The part of Renderer CPU spent submitting buffers and textures, including Godot image staging.
Upload KiB/frame and calls	How much instance data Mold submits and how many upload calls it makes. Includes unused capacity and texture padding.
Updated	Entries accepted by the bulk update, which may include values that did not change. Zero while paused.
Molds / batches / meshes	The current shape count, batch count, and mesh-cache statistics.

LOD CPU and Upload CPU are already included in Renderer CPU, so don't add them again. Renderer CPU covers Mold's work; it is not Godot's entire rendering time, and none of these counters measures GPU execution time.

Frame time uses the elapsed wall-clock time between `_Process` calls. This includes engine work and waits such as VSync. FPS is $1000 / \text{mean frame time in milliseconds}$. The p95 calculation sorts the samples and interpolates between the two nearest entries.

Reading metrics in your own code

Metrics are off by default. Enable them on the context you want to measure:

```
context.PerformanceMetrics.Enabled = true;
MoldPerformanceMetrics metrics = context.PerformanceMetrics;
// Read on the following frame: fields describe the latest completed sync.

context.performance_metrics.enabled = true
var metrics := context.performance_metrics
```

Read the result on the following frame, after Mold has prepared and submitted the previous frame's rendering. `RenderSequence/render_sequence` increases when a measured submission completes, so you can tell whether a new result is available. The performance scene waits for that first result before recording a sample.

Mold reuses the metrics object. If you want to keep a history, copy its numerical fields into your own samples. The values describe the latest measured submission; they do not add together several cameras or submissions. Reading them does not wait for the GPU.

Running a benchmark

Benchmark mode hides the HUD, skips warmup frames, collects samples, then prints one MOLD_BENCHMARK JSON record and exits. Pass the benchmark arguments after Godot's `--` separator:

```
-- --benchmark --benchmark-count=10000 --benchmark-warmup=120 --benchmark-frames=600
```

Add `--benchmark-static` to disable animation. C# also accepts `--benchmark-mode=managed` and `--benchmark-mode=simd` to compare its CPU backends. The result includes both the requested and effective backend, so you can see if SIMD was unavailable.

The sample reserves space for its measurements ahead of time. In interactive mode, formatting, percentile sorting, and statistics queries happen when the HUD refreshes. They are outside the Simulation and Bulk update timers, but still contribute to frame time. Benchmark mode removes that HUD work.

JSON fields

The main frame fields are `frame_mean_ms`, `frame_median_ms`, `frame_p95_ms`, and `fps_from_mean`. `update_mean_ms` and `update_p95_ms` combine Simulation and Bulk update for compatibility with earlier benchmark results. They exclude the later Renderer CPU phase.

The more detailed fields are:

- `simulation_mean_ms`, `simulation_p95_ms`
- `bulk_update_mean_ms`, `bulk_update_p95_ms`
- `render_cpu_mean_ms`, `render_cpu_p95_ms`
- `lod_cpu_mean_ms`, `lod_cpu_p95_ms`
- `upload_cpu_mean_ms`, `upload_cpu_p95_ms`
- `upload_bytes_mean`, `upload_calls_mean`, `updated_molds_mean`
- `port`, `metrics_enabled`, `hud_refresh_hz`, `gpu_time_measured`

The record also includes the workload, renderer, Godot version, VSync mode, and frame cap. Keep these with your results so you know what you compared.

Making a useful comparison

Keep shape count, animation, detail, culling, camera framing, renderer, and build settings the same when comparing a change. Use an exported game with a real graphics device. Headless runs can check that the counters and JSON work, but they cannot tell you what drawing the scene costs on the GPU.

Upload bytes count the instance buffers and textures Mold submits. They exclude mesh creation, uniforms, driver copies, and data written by the GPU. Use an engine or GPU profiler when you need GPU timings, allocations, or memory usage. The [performance guide](#) explains the settings worth comparing.

Blending and render state

Blend modes decide how a shape's color combines with what is already on screen. Mold provides a range of modes for solid shapes, translucent panels, glows, and other effects. Some advanced modes are approximations in Godot, so check how they look in the renderer your game uses.

Choosing a blend mode

Mode	What it does in Godot
Opaque	Draws a solid surface.
Transparent	Mixes the shape with the background using its alpha.
Additive	Adds color to the background, useful for glows.
Multiplicative	Multiplies with the background, useful for darkening.
Subtractive	Subtracts color from the background.
Screen, Lighten, ColorDodge	Use additive blending as an approximation.
LinearBurn, ColorBurn, Darken	Use multiplicative blending as an approximation.
Dither	Fades by drawing fewer pixels in a repeating pattern.

`MoldStyle.Multiply` is also available as an alias for `MoldStyle.Multiplicative`.

By default, `Opaque` and `Dither` write depth. The other built-in modes test against the scene's depth but do not write their own. This lets them blend with what is behind them, although overlapping transparent shapes depend on drawing order.

For reference, ordinary transparent blending is $aC + (1-a)D$, where C is the shape color, D is the background, and a is alpha multiplied by shape coverage. Additive blending is $aC + D$. Godot's spatial shaders provide mix, add, subtract, and multiply operations; the advanced names above do not give you their exact image-editor blend equations.

`Dither` uses a 16×16 Bayer pattern. It can fade while still writing depth, which is useful when ordinary transparency would cause sorting problems. You will see the pattern, especially at lower resolutions.

Dual colors and gradients

Color gradients are chosen separately from the blend mode, so you can use them with opaque, transparent, additive, or other built-in styles.

- `DualRadial` blends by distance from the shape's local origin, including on rims.
- `DualRadialBounds` fits the radial blend to a 2D shape's boundary. On a 3D torus, it blends from the inner edge to the outer edge across radial thickness. Other 3D primitives evaluate it exactly like `DualRadial`.
- `DualAngular` blends around the center of a 2D shape, or around local Y on a 3D primitive, starting at $+X$ and advancing toward $+Z$. Torus arcs fit their start/span range; a full revolution has a color seam. Points on the axis use the first color.
- `DualGradient` blends along a direction across a 2D or 3D shape's bounds.

On 3D primitives, radial distance uses the authored dimensions and is divided by the largest local half-extent, then clamped to 0–1. The hemisphere origin is the center of its base. A sphere is nearly uniform on its curved surface. Object transforms carry these gradients with the shape; gradient direction and space controls apply only to `DualGradient`. Lines retain `Single/DualGradient` support only.

For `DualGradient`, `World` space keeps the direction fixed as the shape rotates. `Object` space rotates the direction with the shape. `World` up is the default, and moving the shape does not shift the gradient through it.

You can choose linear RGB or Oklab color interpolation and add emission. Oklab often gives nicer transitions between colors, but the extra color conversions cost GPU time. Use it where the difference is useful; see the [performance guide](#).

Differences between renderers

Mold converts your colors from sRGB to linear RGB. For an Oklab gradient, it also converts the endpoints to Oklab before storing them, then converts the blended result back to linear RGB in the shader. The `mold_output_color` helper checks Godot's `OUTPUT_IS_SRGB` flag and converts the final output when needed. Editor previews and runtime materials use this same approach in both ports.

Forward+ is the visual reference, and Mobile should look similar. Compatibility uses an LDR/sRGB framebuffer, so HDR emission and advanced blending can look different even with the same style. Ordinary colors and gradients are adjusted to keep their brightness similar, but that does not make every blending result identical.

Depth and sorting

`MoldRenderState` lets you override the style's depth-write default and choose a depth test:

Depth test	When to use it
<code>LessEqual</code>	Normal scene rendering, where closer geometry hides what is behind it.
<code>Greater</code>	Effects that should show where the shape is behind existing geometry.
<code>Always</code>	Overlays that should ignore the scene depth.

These map to Godot's default, inverted, and disabled depth testing. They are the three choices available through Mold's MultiMesh spatial-shader path. Godot controls the underlying depth comparison.

`SortingOrder/sorting_order` sets the material's render priority, clamped to -128 through 127. Use a few consistent orders for layered effects and UI. Godot treats each MultiMesh as a single visual when sorting batches.

Within a batch, C# sorts ordinary `Transparent` instances back to front when depth writes are disabled. It also uses discrete LOD changes for those shapes to avoid splitting one transparent shape across two draws. GDScript currently does not have that per-instance sort or LOD exception.

Face culling and stencil

Face culling is disabled by default, so both sides can render. Choose `MoldFaceCull.Back` or `.Front` in C#, or `MoldRenderState.FaceCull.BACK` or `.FRONT` in GDScript, when you need one-sided rendering.

Stencil lets one shape write a mask and another draw according to that mask. Mold supports reading, writing, and writing when the depth test fails. A shader that reads stencil goes through Godot's transparent pass, even when its Mold style is opaque.

Changing depth, stencil, face culling, or sorting can move a shape to a different batch. Custom materials are a special case: their shaders own the blend, depth, cull, and stencil settings. See [custom materials](#) before using render-state overrides with your own shader.

Custom materials

For the 0.8.0 default, mode switching, native renderer costs, and custom-material exceptions, see [Transparency ordering](#). Existing batching details below apply to Batched and the unchanged opaque/dither paths.

Custom materials let you draw Mold shapes with your own shader. Mold still handles the meshes, transforms, batching, and visibility. You decide how the shapes look.

This feature expects you to be comfortable writing Godot spatial shaders. Mold's built-in effects do not automatically carry over: your shader needs to handle the colors, shape masks, billboards, dashes, and other features you use. Start with a simple 3D shape, then add the behavior your effect needs.

Getting started

1. Write a spatial shader that reads Mold's instance data.
2. Create a `ShaderMaterial` using that shader.
3. Register it with the context that will draw your shapes.
4. Keep the returned material handle and use it in `MoldStyle.Custom/MoldStyle.custom`.
5. Draw shapes with that style, using retained handles or the immediate API.

A material registration belongs to one Mold world. Reuse its handle within that world; register the material again if you need it in a different runtime.

A minimal shader

Mold includes a helper file for reading the data belonging to each shape. The installed path depends on your port:

- C#: `res://addons/mold/Runtime/Rendering/mold_instance_data.gdshaderinc`
- GDScript: `res://addons/mold/runtime/rendering/mold_instance_data.gdshaderinc`

This example colors a mesh and supports continuous LOD transitions. It is a useful starting point for cuboids and spheres. Save it as `res://materials/mold_hologram.gdshader` to use the registration examples below, and change the include path if you are using C#.

```
shader_type spatial;
render_mode unshaded, cull_back;

// Use the path for the installed port.
#include "res://addons/mold/runtime/rendering/mold_instance_data.gdshaderinc"

varying flat vec4 mold_color;
varying flat vec4 mold_lod;

void vertex() {
    mold_color = mold_instance_fetch(
        INSTANCE_ID, MOLD_INSTANCE_PRIMARY_COLOR);
    mold_lod = mold_instance_fetch(
        INSTANCE_ID, MOLD_INSTANCE_LOD_DATA);
}

void fragment() {
    // Required when the world uses Continuous LOD. Without this, both meshes
    // in a transition can be visible at once.
    if (!mold_lod_visible(mold_lod, FRAGCOORD.xy)) {
        discard;
    }

    ALBEDO = mold_output_color(mold_color.rgb);
}
```

`mold_output_color` converts Mold's linear RGB to the output the current Godot renderer expects. Use it for colors you read from Mold. Values that are already in the output color space do not need that conversion again.

For transparency, choose a transparent `render_mode` and write ALPHA. For lighting, remove unshaded and use the normal spatial shader outputs. Mold does not change your shader's render modes for you.

Register and draw

C#

```
using Godot;
using Mold;

public partial class CustomMoldExample : Node3D
{
    private MoldRuntimeInstance _mold = null!;
    private MoldHandle _shape;

    public override void _Ready()
    {
        _mold = MoldRuntime.CreateInstance(this);

        Shader shader = GD.Load<Shader>(
            "res://materials/mold_hologram.gdshader");
        ShaderMaterial material = new() { Shader = shader };

        MoldMaterialHandle materialHandle = _mold.RegisterMaterial(material);
        MoldStyle style = MoldStyle.Custom(materialHandle, Colors.Cyan);

        _shape = _mold.Create(
            MoldShape.Cuboid(new Vector3(2, 2, 2), 0.25f),
            style);
    }

    public override void _ExitTree()
    {
        if (_shape.IsValid)
            _shape.Release();
        _mold.Dispose();
    }
}
```

GDScript

```
extends Node3D

var mold: MoldRuntimeInstance
var shape: MoldHandle

func _ready() -> void:
    mold = MoldRuntime.create_instance(self)

    var shader := load("res://materials/mold_hologram.gdshader") as Shader
    var material := ShaderMaterial.new()
    material.shader = shader

    var material_handle := mold.register_material(material)
    var style := MoldStyle.custom(material_handle, Color.CYAN)
    shape = mold.create(
        MoldShape.cuboid(Vector3(2.0, 2.0, 2.0), 0.25),
        style,
    )

func _exit_tree() -> void:
    if shape and shape.is_valid:
        shape.release()
    if mold:
        mold.dispose()
```

The handle works while its world, registered material, and shader remain valid. Check `IsValid/is_valid` if you keep it in a system that might outlive the runtime. A handle registered with one world cannot be used in another.

Use a custom material in the editor

1. Create a `.gdshader` using the include path for your port.
2. Create a `ShaderMaterial` with that shader and set its shader parameters.
3. Add or select a `MoldNode3D` or `MoldPolylineNode3D`.
4. Expand its `MoldStyleResource` and set **Mode** to **Custom**.
5. Assign **Custom Material** and choose the shape's **Color**.

The node handles registration when it joins a runtime and unregisters when it leaves. It also supplies the instance data for the editor preview. Custom styles currently expose one color, so the dual-color controls are hidden.

Reading instance data

Each shape uses seven floating-point RGBA texture entries, also called texels. Read them with `mold_instance_fetch(INSTANCE_ID, slot)`. Use the constants from the include file so your shader is easier to read.

Slot	Constant	Contents
0	<code>MOLD_INSTANCE_PRIMARY_COLOR</code>	Primary color, with linear RGB and alpha.
1	<code>MOLD_INSTANCE_SECONDARY_COLOR</code>	Secondary color slot; custom styles currently use a single color.
2	<code>MOLD_INSTANCE_SHAPE_DATA</code>	Dimensions and parameters that depend on the shape.
3	<code>MOLD_INSTANCE_DASH_DATA</code>	Dash spacing, profile, and related settings.
4	<code>MOLD_INSTANCE_FLAGS</code>	Emission strength, 2D flag, deformation kind, and packed flags.
5	<code>MOLD_INSTANCE_GRADIENT_DATA</code>	Gradient direction, extent, and mode data.
6	<code>MOLD_INSTANCE_LOD_DATA</code>	LOD coverage, inverted-dither flag, and planar angular range.

The helpers cover the most common operations:

- `mold_instance_fetch(instance_id, slot)` reads one texture entry.
- `mold_local_aa_quality(flags_data)` returns Off (0), Medium (1), or High (2).
- `mold_lod_visible(lod_data, FRAGCOORD.xy)` tells you whether to keep a pixel during a continuous LOD fade.
- `mold_output_color(linear_color)` prepares Mold's RGB for the renderer.

Mold reserves the uniform name `mold_instance_data` and fills it for each batch. Leave that parameter to Mold.

Packed values for shape shaders

You only need these details when implementing the corresponding shape features:

- `flags.w` uses bit 1 to enable local anti-aliasing and bit 32 to choose High quality. Enabled without bit 32 means Medium. Without bit 1, AA is Off.
- Billboard flags use bit 2 for a billboard and bit 16 for the world-Y mode.
- For rounded rectangles and rectangle rims, negative `shape_data.y` means corner-local rounding and stores `-width / height`. Non-negative values use the older shape-relative encoding. Handle both if your shader supports both.
- For rectangles, rectangle rims, regular polygons, and their rims, `lod_data.zw` holds the angular start and span in radians. The full shape uses `vec2(0.0, TAU)`. `lod_data.xy` still holds the LOD fade values.

Keeping shapes in the same batch

Register a material once per world and reuse the handle for shapes that should render together. Registering the same material twice creates separate material identities, so those shapes cannot share a material batch.

Matching handles alone are not enough: the mesh, detail, render state, visual layer, sorting order, and local anti-aliasing also need to match. See [batching](#) if the batch count is higher than you expect.

Mold duplicates your `ShaderMaterial` when a batch is created. Each copy gets its own instance-data texture, which prevents batches from overwriting each other's data.

Set shader parameters before registering the material, then treat that source material as fixed. Changing it later does not update copies that already exist. For a new set of parameter values, register a new material, create a new custom style, and apply it with `SetStyle/set_style`. The shape moves to the new material batch and can keep its mesh.

For values that change frequently, use the supported per-shape data, such as color, or global shader data that your own system manages.

Render-state settings

Your custom shader owns its blend mode, depth test and writes, face culling, lighting, and stencil declarations. Set these in the shader itself.

Mold still applies the visual layer mask to the `MultiMeshInstance3D`, and maps `SortingOrder/sorting_order` to material render priority in the -128 to 127 range. The other render-state fields can split batches, but do not override your shader. If a depth or culling option seems to do nothing, check the shader's `render_mode`.

Supporting more shapes

The minimal shader colors the submitted mesh. To match the built-in appearance, add the parts needed by your chosen shapes:

- **2D shapes:** implement their signed-distance masks. They all use a quad, so without the mask even a disc looks rectangular.
- **Capsules and tori:** apply their vertex deformation data to get the requested dimensions and, for tori, angular span and end caps.
- **Billboards and camera-facing lines:** turn or project the vertices toward the camera.
- **Dashes and thin lines:** add the fragment masks and edge coverage.
- **Polylines:** handle their vertex expansion and stroke coverage. Combine vertex color with the primary instance color for per-point tinting.
- **Continuous LOD:** call `mold_lod_visible` as shown above. Otherwise both meshes can appear at once during a transition. Manual or discrete Automatic LOD avoids that two-mesh fade.

For a more complete starting point, look at `mold_shader.gd` in GDScript. In C#, `MoldEditorPreviewFactory.ShaderSource` in `MoldNode3D.cs` builds the base shader, and `MultiMeshRendering.cs` adapts it for instancing. Keep the shape logic you need and change the shading for your effect.

GPU-polyline materials

Custom materials must opt in with boolean metadata `MoldGpuPolyline = true` and implement the GPU shader contract. C# can start with `MoldGpuPolylineRenderer.CreateShader(mode, state)`; GDScript can assign `MoldGpuPolylineRenderer.shader_source(mode, state)` to a Shader's code. Set the metadata on the `ShaderMaterial`, then use `SetMaterial/set_material` on a stream, or register it as the retained path's custom material. Mold duplicates the material before setting uniforms. The custom shader owns its render modes; changing the stream's render state does not rewrite custom shader source. The shared `mold_gpu_polyline.gdshader.inc` exposes the point/segment texture layout and vertex attribute generator for more extensive customization.

See [GPU polylines](#) for stream setup and backend selection.

Immediate drawing

Custom styles also work with an immediate frame. Register the material on the same context that creates the frame, then reuse its style for the commands you want to draw together. The same mesh and render-state batching rules apply.

Troubleshooting

The shape ignores its color

Check that you read `MOLD_INSTANCE_PRIMARY_COLOR` and pass its RGB through `mold_output_color`. Missing data or applying the color conversion twice can make the shape look white, black, or much brighter than expected.

Every 2D shape looks rectangular

The shader needs a shape mask. A quad is the expected result from the minimal shader above, even when you created a disc or ring.

Shapes double during an LOD transition

Apply `mold_lod_visible` in the fragment shader, or choose Manual or discrete Automatic LOD for that world.

Registration is rejected

Use a valid `ShaderMaterial` with an assigned `Shader`. When drawing, make sure the material handle belongs to the same world as the shape.

Mesh generation and caching

Mold generates meshes when it needs them, then keeps them around for reuse. Many shapes can share one mesh even when their sizes are different. This saves CPU work, but keeping more mesh variants also uses more memory.

What needs a different mesh

All six 2D shape families share a quad. The shader uses each shape's data to draw its outline, corners, rim, and angular span, so those edits do not generate a new mesh.

3D shapes use generated `ArrayMesh` geometry. Their detail level and features such as prism side count or hemisphere caps decide which mesh they need. Rounded cuboids, cylinders, and prisms also account for their proportions so that a round edge stays round after the mesh is scaled.

This means resizing a rounded 3D shape with `SetShape/set_shape` can select a new mesh. `SetScale/set_scale` stretches the existing mesh instead, which is cheaper when you can accept the stretched appearance. Uniform scaling is an easy way to reuse a shape without changing its proportions.

Lines

Rectangle lines place the shared quad between two local XY endpoints. Cylinder lines align a cylinder between two local 3D endpoints. Thickness is the full width of the line, so a cylinder line's radius is half its thickness.

Moving endpoints normally just changes the line's transform. Rounded cylinder lines also account for the ratio between length and thickness when shaping the ends, so a length change can select a different rounded mesh.

Rounded shapes

Rounded cuboids are built from six subdivided faces, with connected rounded edges and corners. The corner radius is shared across all three axes and is limited to a quarter of the shortest dimension. Rounded cylinders and prisms also limit their end rounding to fit the shorter dimension, leaving room for a straight body.

`MoldShape.Detail` controls how many subdivisions these curves use. Higher detail looks smoother up close, at the cost of more vertices and triangles.

Roundness is rounded to a set of cache values rather than keeping a mesh for every possible floating-point value. `RoundnessResolution/roundness_resolution` defaults to 256. Lowering it lets more similar shapes share a mesh; raising it preserves finer differences in their rounded edges.

Spheres, hemispheres, and capsules

Spheres start as an icosahedron and subdivide its triangles. The five detail levels produce 20, 80, 320, 1,280, and 5,120 triangles. This spreads triangles more evenly than a sphere built from latitude rings, which bunch up at the poles.

Hemispheres and capsules use latitude rings because they need a clean equator or separate caps. Capsules share one mesh per detail level. The vertex shader extends the middle while keeping both ends spherical, so each height and radius does not need its own cached mesh.

Tori and billboards

Tori share meshes by detail level, with separate variants for full and partial rings. The shader applies the major radius, tube thickness, and angular range; partial rings also need end-cap geometry. Different partial spans can reuse the same partial-ring mesh.

Billboarded 2D shapes keep the shared quad and turn it toward the camera in the vertex shader. Camera-facing rectangle lines keep their endpoints in place and turn their width toward the camera.

Polylines

CPU polylines generate stroke meshes from their paths. Reusing a path shares its cached geometry. Point edits return a replacement path and generate new geometry. Moving or recoloring the whole handle also reuses the path and mesh.

With Auto, supported paths use GPU expansion instead. For frequently changing points, a GPU stream avoids rebuilding stroke meshes. See [GPU polylines](#).

Managing the cache

Mold keeps meshes that are still in use, plus a limited number of unused meshes in case you need them again.

`RetainedUnusedMeshCount/retained_unused_mesh_count` defaults to 64. When that limit is exceeded, Mold drops the least recently used spare meshes.

Use `MoldContext.Prewarm(...)/prewarm(...)` before a busy frame to generate the shapes you expect to need. Prewarmed meshes are still subject to the unused-cache limit. `ClearUnusedMeshes()/clear_unused_meshes()` drops meshes that no shape currently uses.

Mesh-generation work is adapted from Keijiro Takahashi's MetaMesh. See the [third-party notices](#) for attribution.

GPU polylines

Use `MoldGpuPolylineRenderer` for paths whose points change frequently. Create the paths with `SetData/set_data`, animate with `UpdatePoints/update_points`, and dispose the stream when finished. Disposing the owning runtime also disposes its streams.

The Godot implementation uses a fixed `ArrayMesh` and two floating-point textures. Its vertex shader expands neighboring points into stroke geometry, then feeds Godot's existing polyline coverage/color shader. Point updates upload a texture; they do not regenerate or upload a stroke mesh. There is no compute dispatch, `RenderingDevice` requirement, or `GDEXTension` dependency.

Forward+, Mobile, and Compatibility are supported. `IsSupported/is_supported()` returns false in headless mode. Desktop rendering has been tested on all three renderers; browser exports and other GPU/driver combinations still need target validation. C# retains Godot's existing .NET web-export limitation.

Retained and authored paths

`MoldContext.Create(polyline, ..., backend: MoldPolylineBackend.Auto)` and `create_polyline(..., MoldPolylineBackend.Mode.AUTO)` use the GPU backend when eligible.

`MoldPolylineNode3D.Backend/backend` uses the same world policy.

`IsUsingGpuBackend/is_using_gpu_backend` reports the node's actual allocation. Editor previews continue using CPU meshes.

Explicit CPU always uses cached geometry. Auto selects GPU expansion whenever the backend supports the path and material. This includes multiple retained handles sharing the same immutable path object. It falls back to CPU when GPU rendering is unavailable, the point count exceeds stream capacity, the color mode is unsupported, or a custom material has not opted into the GPU shader contract. If eligibility changes, Mold switches backends before the next rendering update. The handle stays valid. Each GPU path currently owns a separate draw.

`SetPolyline`, `SetPolylinePoint`, and `SetPolylinePoints` keep their create-and-replace path behavior. When point count and closed/open topology stay unchanged, a GPU handle reuses its carrier mesh and uploads new point data. These convenience setters still allocate replacement path objects. Use the explicit stream API for large animated datasets.

Create and update a stream

C#

Keep the runtime, stream, and point array in fields. The setup belongs in `_Ready`, point updates in your animation method, and disposal in `_ExitTree`. Use `using Godot;` and `using Mold;` in the script.

```
MoldRuntimeInstance runtime = MoldRuntime.CreateInstance(this);
MoldGpuPolylineRenderer stream = runtime.CreateGpuPolylineRenderer();
MoldGpuPolylinePoint[] points =
[
    new(new Vector3(-1, 0, 0), 0.05f, Colors.Cyan),
    new(new Vector3( 0, 1, 0), 0.08f, Colors.White),
    new(new Vector3( 1, 0, 0), 0.05f, Colors.Magenta),
];
stream.SetData(points, [new MoldGpuPolylineRange(0, points.Length)]);
stream.Join = MoldPolylineJoin.Round;
stream.Cap = MoldPolylineCap.Round;
stream.Configure(MoldStyle.Transparent(Colors.White));

// In an animation update: reuse your point array.
points[1].Position.Y = animatedHeight;
stream.UpdatePoints(points);

// On teardown:
stream.Dispose();
runtime.Dispose();
```

GDScript

GDScript uses eight packed floats per point to avoid one object allocation per animated point. The fields are **x**, **y**, **z**, **absolute thickness**, **r**, **g**, **b**, **a**.

```
var runtime := MoldRuntime.create_instance(self)
var stream := runtime.create_gpu_polyline_renderer()
var points := PackedFloat32Array([
    -1, 0, 0, 0.05, 0, 1, 1, 1,
    0, 1, 0, 0.08, 1, 1, 1, 1,
    1, 0, 0, 0.05, 1, 0, 1, 1,
])
stream.set_data(points, [MoldGpuPolylineRange.new(0, 3)])
stream.join = MoldPolyline.Join.ROUND
stream.cap = MoldPolyline.Cap.ROUND
stream.configure(MoldStyle.transparent(Color.WHITE))

points[9] = animated_height
stream.update_points(points)

stream.dispose()
runtime.dispose()
```

Points use absolute stroke thickness; `MoldPolylinePoint` instead multiplies a path's base thickness.

`SetPolyline/set_polyline` performs that conversion. Ranges may describe multiple open or closed paths. Open paths require at least two points; closed paths require three. The stream shares one transform, style, render state, and sorting order across its ranges. Points and thickness must be finite, and thickness positive. Keep those values valid on every update; initial validation does not make later point data safe automatically.

`SetTransform/set_transform` sets the transform relative to the owning world. `SetVisible/set_visible` controls the whole stream. Points may preserve a Z coordinate, but joins and width are calculated in local XY; this is a planar stroke renderer, not a camera-facing 3D ribbon or tube renderer.

Bounds, materials, and costs

Updates recalculate conservative CPU bounds by default. Supply a fixed animation envelope with `SetLocalBounds/set_local_bounds` and pass `false` to point updates to skip that scan. Include the full animated stroke, joins, caps, and anticipated AA expansion in manual bounds. Automatic bounds include stroke/miter padding and an extra local-space margin; extreme zoom-out can require a larger manual bound.

Miter, bevel, and round joins; butt, square, and round caps; per-point colors and widths; Single/DualGradient colors; and the existing Godot blend/depth/stencil behavior are supported. GPU point colors retain float/HDR precision; CPU mesh vertex colors remain subject to Godot's vertex-color quantization. Existing Godot blend-mode approximations still apply.

Custom materials must opt in with boolean metadata `MoldGpuPolyline = true` and implement the GPU shader contract. C# can start with `MoldGpuPolylineRenderer.CreateShader(mode, state)`; GDScript can assign `MoldGpuPolylineRenderer.shader_source(mode, state)` to a Shader's code. Set the metadata on the `ShaderMaterial`, then use `SetMaterial/set_material` on a stream, or register it as the retained path's custom material. Mold duplicates the material before setting uniforms. The custom shader owns its render modes; changing the stream's render state does not rewrite custom shader source. The shared `mold_gpu_polyline.gdshaderinc` exposes the point/segment texture layout and vertex attribute generator for more extensive customization.

`SetData` records the distances along each path. Point uploads therefore preserve rest-length progress; call `SetData` if a custom shader needs new distance metrics. Public polyline dashes remain unsupported. Round subdivisions do not affect the stroke outline.

The carrier uses 12 vertices per segment for miter joins without round caps, otherwise 36. The extra vertices cover the joins and caps. A change between these layouts rebuilds the carrier; ordinary point motion does not.

Point payload is 32 bytes per point, plus texture row padding and engine copies. Textures use 1024-wide rows with a maximum of 4096 rows per stream; split larger streams. `ImageTexture` updates upload a whole image. GDScript packed-array copy and byte conversion costs remain, so CPU point production and upload time should be measured separately from GPU expansion. Shared retained paths each receive a GPU stream under `Auto`.

Try the example

Open the [C# demo](#) or [GDScript demo](#). Both show an animated wave field and provide Auto/CPU buttons, an animation toggle, a line-count control from 8 to 1024, and frame, point-production, and update timing readouts. They start at 128 paths with 192 points each. Topology changes only when Set is pressed or relevant exported settings change. The timings include normal frame overhead and are not GPU timer queries.

Rendering game UI with Mold

Mold is useful for UI made from animated shapes: health bars, cooldown rings, reticles, minimaps, markers, and selection outlines. Create the shapes once, keep their handles, and update them as the game changes.

Use Godot's `CanvasLayer` and `Control` nodes for text, buttons, input, focus, accessibility, and automatic layout. Mold draws in 3D space, so the usual setup is a flat layer of shapes facing an orthographic camera, with native controls placed over it.

Start with the showcase

The [C#](#) and [GDScript](#) UI showcases are runnable examples. They include a speedometer, motion tracker, navigation compass, and radial weapon selector. Mold draws their plates, ticks, rings, icons, and fills, while native `Label` controls display the changing text.

The moving 3D scene behind the HUD renders into a separate `SubViewport`. Its texture appears on a quad behind the UI. This lets the gameplay camera move freely while the HUD keeps a steady coordinate system.

Set up the camera and coordinates

For a simple HUD, start with this scene structure:

```
HudRoot (Node3D, owns the Mold runtime)
├─ Camera3D (orthogonal, current)
└─ Interface (CanvasLayer)
    └─ LabelsAndInput (Control)
```

Use one coordinate system for both shapes and controls. A 16×9 canvas is convenient: $(0, 0)$ is the top-left and $(16, 9)$ is the bottom-right. The following helper centers that canvas at the world origin and flips Y, since UI coordinates run downward while Godot's world Y runs upward:

```
private const float CanvasWidth = 16f;
private const float CanvasHeight = 9f;

private static Vector3 UiPoint(float x, float y) =>
    new(x - CanvasWidth * 0.5f, CanvasHeight * 0.5f - y, 0f);

const CANVAS_WIDTH := 16.0
const CANVAS_HEIGHT := 9.0

func ui_point(x: float, y: float) -> Vector3:
    return Vector3(x - CANVAS_WIDTH * 0.5, CANVAS_HEIGHT * 0.5 - y, 0.0)
```

Place the camera at $(0, 0, 10)$, looking along its default -Z direction. Make it current, choose orthogonal projection, and set its size to 9. With the default Keep Height setting, this shows the whole canvas at 16:9.

Wider displays reveal more space at the sides. On narrower displays, either letterbox the viewport or adjust the layout to fit the available area.

Create the shapes once

Create an owned runtime when the HUD enters the tree and keep handles for its visual elements. Manual LOD is a useful choice here because the HUD's screen size is predictable.

These snippets belong in a `Node3D` script attached to `HudRoot`. Use the coordinate helper above; [C#](#) also needs `using Godot;` and `using Mold;`.

```
private MoldRuntimeInstance _ui = null!;
private MoldHandle _healthFill;

public override void _Ready()
{
    Camera3D camera = GetNode<Camera3D>("Camera3D");
    camera.Projection = Camera3D.ProjectionType.Orthogonal;
    camera.Size = CanvasHeight;
    camera.Position = new(0f, 0f, 10f);
```

```

_ui = MoldRuntime.CreateInstance(this, new MoldWorldSettings
{
    Capacity = 256,
    LodMode = MoldLodMode.Manual,
    RetainedUnusedMeshCount = 32,
});

_healthFill = _ui.Create(
    MoldShape.Rectangle(new Vector2(3.2f, 0.22f), 0.45f),
    MoldStyle.DualGradient(
        new Color("55e6a5"), new Color("b4ff82"), Vector3.Right,
        MoldGradientSpace.Object, MoldStyle.BlendMode.Opaque,
        MoldColorInterpolation.Oklab),
    UiState(20));
_healthFill.SetPosition(UiPoint(2.2f, 1.1f));
}

private static MoldRenderState UiState(int order) => new(
    SortingOrder: order,
    DepthTest: MoldDepthTest.Always,
    DepthWrite: MoldDepthWrite.Disabled,
    FaceCull: MoldFaceCull.Disabled);

```

```

var ui: MoldRuntimeInstance
var health_fill: MoldHandle

func _ready() -> void:
    var camera: Camera3D = $Camera3D
    camera.projection = Camera3D.PROJECTION_ORTHOGONAL
    camera.size = CANVAS_HEIGHT
    camera.position = Vector3(0.0, 0.0, 10.0)

    var settings := MoldWorldSettings.new()
    settings.capacity = 256
    settings.lod_mode = MoldWorldSettings.LodMode.MANUAL
    settings.retained_unused_mesh_count = 32
    ui = MoldRuntime.create_instance(self, settings)

    health_fill = ui.create(
        MoldShape.rectangle(Vector2(3.2, 0.22), 0.45),
        MoldStyle.dual_gradient(
            Color("55e6a5"), Color("b4ff82"), Vector3.RIGHT,
            MoldStyle.GradientSpace.OBJECT, MoldStyle.BlendMode.OPAQUE,
            MoldStyle.ColorInterpolation.OKLAB),
        ui_state(20)
    )
    health_fill.set_position(ui_point(2.2, 1.1))

func ui_state(order: int) -> MoldRenderState:
    return MoldRenderState.new(
        1, order, MoldRenderState.DepthTest.ALWAYS,
        MoldRenderState.DepthWrite.DISABLED,
        MoldRenderState.StencilFlags.DISABLED,
        MoldRenderState.StencilCompare.ALWAYS, 0,
        MoldRenderState.FaceCull.DISABLED)

```

`DepthTest.Always` lets the HUD draw regardless of scene depth. Disabling depth writes prevents one UI shape from blocking later layers through the depth buffer. Disabling face culling lets the shapes remain visible from either side.

Use a few consistent sorting orders, such as background - 100, panels 0, values 10, icons 20, and alerts 100. Different orders and blend modes can split batches, so there is little reason to give every element a unique state.

These shapes are useful starting points:

UI element	Mold geometry
Panel or filled bar	Rectangle
Border or focus outline	RectangleRim
Cooldown or objective progress	Ring with an angular span
Reticle, badge, or marker	Disc, Ring, or RegularPolygon
Divider or tick	Line2D

UI element	Mold geometry
Minimap route or graph	MoldPolyline
Dashed radar or selection outline	MoldDash on a ring or rim

Update the handles

During `_Process`, update the values that changed. For a bar, change its width and move its center so the left edge stays in place:

```
private void SetBar(MoldHandle handle, float left, float y, float fullWidth,
    float height, float value)
{
    float width = fullWidth * Mathf.Clamp(value, 0f, 1f);
    handle.SetShape(MoldShape.Rectangle(new Vector2(width, height), 0.45f));
    handle.SetPosition(UiPoint(left + width * 0.5f, y));
}

func set_bar(handle: MoldHandle, left: float, y: float,
    full_width: float, height: float, value: float) -> void:
    var width := full_width * clampf(value, 0.0, 1.0)
    handle.set_shape(MoldShape.rectangle(Vector2(width, height), 0.45))
    handle.set_position(ui_point(left + width * 0.5, y))
```

For a cooldown, keep the ring's radius and change its angular span. For a route, replace the path with `SetPolyline/set_polyline`. If many markers move at once, use `UpdatePositions/update_positions_bulk`.

Keep the handles instead of releasing and recreating them every frame. Mold can reuse their meshes and batches where possible. Store the values you need for later edits, such as a ring's radius, thickness, and dash settings, beside the handle.

Keep native text aligned

Create `Label`, `Button`, and other controls below a `CanvasLayer`. Use the HUD camera to convert your logical coordinates into screen positions:

```
Vector2 screen = camera.UnprojectPosition(UiPoint(x, y));
label.Position = screen;

var screen := camera.unproject_position(ui_point(x, y))
label.position = screen
```

Update these positions when the camera or viewport changes. Project two points one logical unit apart to find the pixel scale for font sizes and control bounds.

Native controls give you font shaping, localization, keyboard focus, accessibility, clipping, and input handling. A button can use a `Control` for its clickable area while Mold draws its background. Mold shapes do not inherit `Control` anchors, containers, themes, clipping masks, or mouse filtering, so keep that layout and interaction code in the native UI layer.

Cleanup and world-space UI

Release the handles and dispose your owned runtime in `_ExitTree`. Disposing the runtime also clears the shapes it owns. If you connect `resize` or other signals, disconnect them when the HUD leaves the tree.

For a panel that belongs in the game world, use the same create-once and update approach, but keep the normal depth test so scenery can hide it. Project its text anchors through the gameplay camera if the text still uses screen controls.

Rendering backends

For the 0.8.0 default, mode switching, native renderer costs, and custom-material exceptions, see [Transparency ordering](#). Existing batching details below apply to Batched and the unchanged opaque/dither paths.

Mold uses the renderer selected by your project. Check colors, transparency, and masks in the renderer and build you plan to ship.

Choose a renderer

Forward+, Mobile, and Compatibility are supported. Forward+ is the visual reference, and Mobile should look similar. Compatibility uses an LDR/sRGB framebuffer, so HDR emission and advanced blending can look different.

Both language ports use the same shape and color approach. C# requires Godot .NET; use GDScript for web exports.

Colors and effects

Ordinary colors and gradients are adjusted for the renderer's output color space. This keeps their brightness similar, but does not make every blend result identical. Some advanced blend modes use approximations in spatial shaders. See [blending](#) for the supported behavior.

Depth and stencil

The MultiMesh spatial-shader path supports `LessEqual`, `Greater`, and `Always` depth tests. Stencil supports reading, writing, and depth-fail writes. A stencil reader uses the transparent pass, including for otherwise opaque shapes.

Custom shaders own their depth, blend, face-culling, and stencil declarations. See [custom materials](#) before overriding their render state.

UI and limitations

Mold draws in 3D space. Use a camera-aligned plane for HUD shapes and native `CanvasLayer/Control` nodes for text and interaction. See [UI rendering](#) for the setup.

GPU polylines work through textures and spatial shaders without a `RenderingDevice` dependency. Headless mode cannot use that backend. Test browser exports and your target GPU separately; see [GPU polylines](#).

Mold Godot for GDScript

Version 0.8.0 defaults to NativeCompatible transparency. The Performance Test scene switches between Native Compatible and Batched on a 75%-alpha workload. See [Transparency ordering](#) for behavior, costs, and upgrade notes.

This guide covers creating and updating Mold shapes from GDScript. For installation, requirements, and the full list of example scenes, start with the [main README](#). This directory is also a runnable Godot project, so you can open it and try the examples directly.

Create a runtime you own

For most procedural systems, create a `MoldRuntimeInstance` and keep it for as long as that system needs to draw. You control its settings, mesh cache, and cleanup. The following script creates a rounded cuboid and places it one unit above the origin:

```
extends Node3D

var mold: MoldRuntimeInstance
var handles: Array[MoldHandle] = []

func _ready() -> void:
    mold = MoldRuntime.create_instance(self)
    var shape := mold.create(
        MoldShape.cuboid(Vector3(2.0, 2.0, 2.0), 0.4),
        MoldStyle.dual_gradient(Color.CYAN, Color.BLUE),
    )
    shape.set_position(Vector3(0.0, 1.0, 0.0))
    handles.append(shape)

func _exit_tree() -> void:
    for handle in handles:
        if handle.is_valid:
            handle.release()
    mold.dispose()
```

Attach this script to a `Node3D` in a scene with an active `Camera3D` positioned to see the shape. Creating the shape returns a handle; keep that handle to move, recolor, resize, or release it later.

Call `dispose()` when your system is done. It clears the runtime and the shapes it owns. Mold also cleans up if the scene or `SubViewport` goes away, but disposing it yourself makes the lifetime clear.

The later snippets go inside functions on this script and use the same `mold` runtime. Keep any handles you need to update later in `handles` or their own member variables.

Shared and automatic runtimes

For a small script or quick prototype, the shared runtime is shorter:

```
var shape := MoldRuntime.create(
    MoldShape.cuboid(Vector3(2.0, 2.0, 2.0), 0.4),
    MoldStyle.dual_gradient(Color.CYAN, Color.BLUE),
)
```

If you need custom settings, call `MoldRuntime.configure_shared(settings)` before anything uses it. The shared runtime draws in the root viewport and survives scene changes. Every caller shares its capacity, mesh cache, and immediate frame, so remember to release every retained handle you create there.

`MoldRuntime.for_node(self, settings)` is another option. It shares a runtime within the current scene or `SubViewport`. The first call chooses its settings; later calls reuse it and reject conflicting settings. Its handles stop being valid when that scene or viewport leaves the tree.

This automatic runtime is what `MoldNode3D` and `MoldPolylineNode3D` use. For a procedural system that can own its runtime, `MoldRuntime.create_instance(self)` is usually the easier choice to manage.

Lines and rounded shapes

`line_2d` draws a flat rectangle between endpoints, while `line_3d` draws a cylinder. Thickness is the full width of the line. Move the endpoints through the handle when you want to change its length or direction:

```
var handle := mold.create(
    MoldShape.line_3d(Vector3.ZERO, Vector3.UP * 2.0, 0.12, 0.35),
    MoldStyle.opaque(Color.WHITE),
)
handle.set_line_positions(Vector3.ZERO, Vector3.RIGHT * 3.0)
```

Both line types support roundness. For a rounded `line_2d`, pass the billboard mode after thickness, then roundness.

Rectangle lines default to `CORNER_LOCAL` rounding, which keeps the caps round. Use `SHAPE_RELATIVE` if you want the rounding to stretch with the line. The same controls are available on Rectangle resources in the Inspector when line mode is enabled.

Rounded cuboids, cylinders, and prisms keep their edge radius consistent across the shape, with limits based on the shortest dimension. Changing their proportions, including the length of a rounded cylinder line, can select another cached mesh. See [mesh generation](#) for when scaling can save that work.

Polylines

A `MoldPolyline` draws a connected stroke in its local XY plane. Each point can have its own color and thickness multiplier. Paths can be open or closed, with miter, bevel, or round joins and butt, square, or round end caps.

```
var path := MoldPolyline.new([
    MoldPolylinePoint.new(Vector3.LEFT, Color.RED),
    MoldPolylinePoint.new(Vector3.UP, Color.WHITE, 1.5),
    MoldPolylinePoint.new(Vector3.RIGHT, Color.BLUE),
], 0.08, false, MoldPolyline.Join.ROUND, MoldPolyline.Cap.ROUND,
4.0, 8, MoldDash.new(8, 12, 0.25))

var path_handle := mold.create_polyline(
    path, MoldStyle.transparent(Color.WHITE))
path_handle.set_polyline_point(
    1, MoldPolylinePoint.new(Vector3.UP * 1.5, Color.GREEN, 1.5))
```

Reuse the same `MoldPolyline` for unchanged paths so shapes can share its mesh. `set_polyline`, `set_polyline_point`, and `set_polyline_points` replace the handle's path; they do not edit the original path object. Moving the whole handle keeps its mesh. Immediate drawing also supports `draw.polyline(path)`.

To author a path in the editor, add `MoldPolylineNode3D`, create its `MoldPolylineResource`, and edit the nested points in the Inspector. The node provides a preview and handles the runtime registration for you.

GPU polylines

Animated paths can use `MoldGpuPolylineRenderer` to update points without rebuilding a stroke mesh. The polyline backend policy routes Auto paths to GPU expansion whenever it is supported, and otherwise falls back to cached CPU geometry. See [GPU polylines](#) for the API and limitations, and [Polyline Backend](#) for the interactive CPU/GPU comparison demo.

Partial shapes and billboards

A torus can use an angular span to draw part of a ring. A 2D shape can face the camera, or turn only around world Y to stay upright, like a Doom sprite:

```
var torus := mold.create(
    MoldShape.torus(1.0, 0.25, 0.0, PI * 1.5),
    MoldStyle.dual_gradient(Color.VIOLET, Color.CYAN),
)
var badge := mold.create(
    MoldShape.disc(0.5).with_billboard(),
    MoldStyle.transparent(Color.WHITE),
)
var upright_badge := mold.create(
```

```

    MoldShape.disc(0.5).with_billboard(MoldShape.BillboardMode.FACE_CAMERA_Y),
    MoldStyle.transparent(Color.WHITE),
)
var view_line := mold.create(
    MoldShape.line_2d(
        Vector3(-1.0, 0.0, -1.0), Vector3(1.0, 0.0, 1.0), 0.1,
        MoldShape.BillboardMode.FACE_CAMERA,
    ),
    MoldStyle.opaque(Color.WHITE),
)

```

The two badges show the full camera-facing and upright options. The final line keeps its endpoints in 3D space and turns its width toward the camera.

Colors and gradients

`dual_radial()` blends by distance from the local origin of a 2D or 3D shape, including rims. Use `dual_radial_bounds()` to fit the gradient to the shape's boundary:

```

var centered := MoldStyle.dual_radial(Color.WHITE, Color.BLUE)
var fitted := MoldStyle.dual_radial_bounds(Color.WHITE, Color.BLUE)

```

`dual_radial_bounds()` blends from the inner edge to the outer edge on a 3D torus; other 3D primitives treat it as radial. Angular color rotates around local Y from +X toward +Z, fitting a torus's start/span range. Directional gradients also work on 3D shapes. You can choose linear RGB or Oklab interpolation separately from the blend mode. Oklab can give nicer color transitions, but costs extra GPU work. See [blending](#) for the modes and renderer differences.

Dashes

Dashes work on rings, rims, and lines. The count-based pattern below fits 12 periods around the ring, with gaps taking up 45% of each period:

```

var dash := MoldDash.new(
    12,
    12,
    0.45,
    0.0,
    MoldDash.Type.CHEVRON,
    0.8,
)

var ring := mold.create(
    MoldShape.ring(1.5, 0.12, 0.0, TAU, dash),
    MoldStyle.additive(Color.CYAN),
)

var dashed_line := MoldShape.line_3d(
    Vector3.LEFT, Vector3.RIGHT, 0.12, 0.35,
).with_dash(MoldDash.new(10, 10, 0.5))

```

For a line, a count-based pattern stretches across the endpoint span. Fixed-length dash patterns are also available when you want the dash and gap lengths to stay in local-space units.

Flat rectangle lines support normal, chevron, and rounded dash profiles. Cylinder lines use the normal profile. Their gaps are shader cuts, so there are no extra cap faces inside those gaps.

Update many shapes at once

Use the bulk APIs when many shapes change together. Reuse the input arrays across frames to avoid creating more temporary data.

```

var positions := PackedVector3Array()
positions.resize(handles.size())

for index in handles.size():
    positions[index] = calculate_position(index)

mold.update_positions_bulk(handles, positions)

```

`calculate_position(index)` stands for your own animation function. Allocate `positions` once and refill it when updating the shapes. The other bulk methods are:

Change	Method and input arrays
Colors	<code>update_colors_bulk(handles, colors)</code> with a <code>PackedColorArray</code> .
Full transforms	<code>update_transforms_bulk(handles, positions, rotations, scales)</code> with packed position/scale arrays and <code>Array[Quaternion]</code> rotations.
Line endpoints	<code>update_line_positions_bulk(handles, starts, ends)</code> with two <code>PackedVector3Array</code> values.

All arrays in a call must have equal lengths. Mold reads them during the call, skips invalid handles, and uses the last value if you include a handle more than once. The return value counts accepted entries, including duplicates; endpoint updates also skip handles that are not lines. Finish any worker writes before passing the arrays to Mold.

Use position-only updates when rotation and scale are unchanged. Color-only updates likewise avoid touching the transform. See [batching](#) for how these edits reach the renderer.

Immediate drawing

For content you want to describe again every frame, the immediate API saves you from keeping a handle for each shape. `draw()` replaces the previous immediate frame for that context. `with_state()` restores your drawing settings when the callback ends:

```
mold.draw(func(draw: MoldImmediate) -> void:
    draw.with_state(func() -> void:
        draw.style = MoldStyle.opaque(Color.CYAN)
        draw.thickness = 0.04
        draw.dash = MoldDash.new(10, 10, 0.5)
        draw.line(Vector3.LEFT, Vector3.RIGHT)
    )
)
```

Run this drawing code each frame if the content is animated. Commands stay visible until the next immediate frame replaces them or the runtime shuts down. Keep immediate drawing coordinated within a context, since starting another frame replaces what earlier callers drew.

For camera-dependent drawing and editor visualization, extend `MoldImmediateDrawer3D` from a tool script. Mold invokes all drawers in a viewport through one shared frame, so compatible commands still batch together:

```
@tool
extends MoldImmediateDrawer3D

@export_range(0.0, 60.0, 0.1) var preview_time := 0.0:
    set(value):
        preview_time = value
        request_redraw()

func draw_molds(draw: MoldImmediate, camera: Camera3D) -> void:
    draw.style = MoldStyle.opaque(Color.CYAN)
    draw.line(Vector3.ZERO, Vector3.FORWARD * 2.0)
```

Call `request_redraw()` after externally owned preview data changes. Editor cameras automatically trigger another frame when their transform, projection, viewport, or culling mask changes. Editor previews use each shape's requested detail exactly; runtime drawers use the normal screen-space LOD policy. Enable `continuously_redraw` only for editor animation that changes without an Inspector or transform edit.

Mesh cache and statistics

Prewarm shapes before a busy frame if you want to generate their meshes ahead of time. You can also read the context's statistics to see how many shapes and batches it currently holds:

```
mold.prewarm([
    MoldShape.sphere(1.0, MoldShape.Detail.MEDIUM),
    MoldShape.cuboid(Vector3.ONE, 0.35),
])

var stats := mold.statistics
print("%d molds in %d batches" % [stats.active_mold_count, stats.batch_count])
```

`active_mold_count` counts retained shapes. Use `immediate_command_count` and `immediate_batch_count` for immediate drawing. `batch_count` includes both kinds. Call `clear_unused_meshes()` when you want to drop cached meshes that are no longer in use.

Prewarmed meshes still count toward the unused-cache limit. See [mesh generation](#) for the cache settings.

More guides

- [Performance choices](#)
- [Performance test metrics](#)
- [Custom materials](#)
- [Rendering game UI](#)
- [Architecture and runtime ownership](#)

Mold Godot for C#

Version 0.8.0 defaults to NativeCompatible transparency. The Performance Test scene switches between Native Compatible and Batched on a 75%-alpha workload. See [Transparency ordering](#) for behavior, costs, and upgrade notes.

This guide covers creating and updating Mold shapes from C#. For installation, requirements, and the full list of example scenes, start with the [main README](#). This directory is also a runnable Godot project, so you can open it and try the examples directly.

Use the .NET build of Godot and build the project before running the C# scenes.

Create a runtime you own

For most procedural systems, create a `MoldRuntimeInstance` and keep it for as long as that system needs to draw. You control its settings, mesh cache, and cleanup. The following script creates a rounded cuboid and places it one unit above the origin:

```
using System.Collections.Generic;
using Godot;
using Mold;

public partial class MoldExample : Node3D
{
    private MoldRuntimeInstance _mold = null!;
    private readonly List<MoldHandle> _handles = [];

    public override void _Ready()
    {
        _mold = MoldRuntime.CreateInstance(this);
        MoldHandle shape = _mold.Create(
            MoldShape.Cuboid(new Vector3(2, 2, 2), 0.4f),
            MoldStyle.DualGradient(Colors.Cyan, Colors.Blue));
        shape.SetPosition(new Vector3(0, 1, 0));
        _handles.Add(shape);
    }

    public override void _ExitTree()
    {
        foreach (MoldHandle handle in _handles)
            if (handle.IsValid)
                handle.Release();
        _mold.Dispose();
    }
}
```

Attach this script to a `Node3D` in a scene with an active `Camera3D` positioned to see the shape. Creating the shape returns a handle; keep that handle to move, recolor, resize, or release it later.

Call `Dispose()` when your system is done. It clears the runtime and the shapes it owns. Mold also cleans up if the scene or `SubViewport` goes away, but disposing it yourself makes the lifetime clear.

The later snippets go inside methods on this script and use the same `_mold` runtime. Add any retained handles you need to update later to `_handles` or keep them in their own fields.

Shared and automatic runtimes

For a small script or quick prototype, the shared runtime is shorter:

```
MoldHandle shape = MoldRuntime.Create(
    MoldShape.Cuboid(new Vector3(2, 2, 2), 0.4f),
    MoldStyle.DualGradient(Colors.Cyan, Colors.Blue));
```

If you need custom settings, call `MoldRuntime.ConfigureShared(settings)` before anything uses it. The shared runtime draws in the root viewport and survives scene changes. Every caller shares its capacity, mesh cache, and immediate frame, so remember to release every retained handle you create there.

`MoldRuntime.For(this, settings)` is another option. It shares a runtime within the current scene or `SubViewport`. The first call chooses its settings; later calls reuse it and reject conflicting settings. Its handles stop being valid when that scene or viewport leaves the tree.

This automatic runtime is what `MoldNode3D` and `MoldPolylineNode3D` use. For a procedural system that can own its runtime, `MoldRuntime.CreateInstance(this)` is usually the easier choice to manage.

Lines and rounded shapes

`Line2D` draws a flat rectangle between endpoints, while `Line3D` draws a cylinder. Thickness is the full width of the line. Move the endpoints through the handle when you want to change its length or direction:

```
MoldHandle handle = _mold.Create(
    MoldShape.Line3D(Vector3.Zero, Vector3.Up * 2.0f, 0.12f, 0.35f),
    MoldStyle.Opaque(Colors.White));
handle.SetLinePositions(Vector3.Zero, Vector3.Right * 3.0f);
```

Both line types support roundness. For a rounded `Line2D`, pass roundness after the diameter; the optional billboard mode comes after roundness.

Rectangle lines default to `CornerLocal` rounding, which keeps the caps round. Use `ShapeRelative` if you want the rounding to stretch with the line. The same controls are available on `Rectangle` resources in the Inspector when line mode is enabled.

Rounded cuboids, cylinders, and prisms keep their edge radius consistent across the shape, with limits based on the shortest dimension. Changing their proportions, including the length of a rounded cylinder line, can select another cached mesh. See [mesh generation](#) for when scaling can save that work.

Polylines

A `MoldPolyline` draws a connected stroke in its local XY plane. Each point can have its own color and thickness multiplier. Paths can be open or closed, with miter, bevel, or round joins and butt, square, or round end caps.

```
MoldPolyline path = new(
    [
        new(Vector3.Left, Colors.Red),
        new(Vector3.Up, Colors.White, 1.5f),
        new(Vector3.Right, Colors.Blue),
    ],
    thickness: 0.08f,
    join: MoldPolylineJoin.Round,
    cap: MoldPolylineCap.Round,
    dash: new MoldDash(8, 12, 0.25f));

MoldHandle pathHandle = _mold.Create(path, MoldStyle.Transparent(Colors.White));
pathHandle.SetPolylinePoint(1,
    new MoldPolylinePoint(Vector3.Up * 1.5f, Colors.Green, 1.5f));
```

Reuse the same `MoldPolyline` for unchanged paths so shapes can share its mesh. `SetPolyline`, `SetPolylinePoint`, and `SetPolylinePoints` replace the handle's path; they do not edit the original path object. Moving the whole handle keeps its mesh. Immediate drawing also supports `draw.Polyline(path)`.

To author a path in the editor, add `MoldPolylineNode3D`, create its `MoldPolylineResource`, and edit the nested points in the Inspector. The node provides a preview and handles the runtime registration for you.

GPU polylines

Animated paths can use `MoldGpuPolylineRenderer` to update points without rebuilding a stroke mesh. The polyline backend policy routes `Auto` paths to GPU expansion whenever it is supported, and otherwise falls back to cached CPU geometry. See [GPU polylines](#) for the API and limitations, and [Polyline Backend](#) for the interactive CPU/GPU comparison demo.

Partial shapes and billboards

A torus can use an angular span to draw part of a ring. A 2D shape can face the camera, or turn only around world Y to stay upright, like a Doom sprite:

```
MoldHandle torus = _mold.Create(
    MoldShape.Torus(1.0f, 0.25f, 0.0f, Mathf.Pi * 1.5f),
    MoldStyle.DualGradient(Colors.Violet, Colors.Cyan));
MoldHandle badge = _mold.Create(
    MoldShape.Disc(0.5f).WithBillboard(),
    MoldStyle.Transparent(Colors.White));
MoldHandle uprightBadge = _mold.Create(
    MoldShape.Disc(0.5f).WithBillboard(MoldShape.BillboardMode.FaceCameraY),
    MoldStyle.Transparent(Colors.White));
MoldHandle viewLine = _mold.Create(
    MoldShape.Line2D(
        new(-1, 0, -1), new(1, 0, 1), 0.1f,
        MoldShape.BillboardMode.FaceCamera),
    MoldStyle.Opaque(Colors.White));
```

The two badges show the full camera-facing and upright options. The final line keeps its endpoints in 3D space and turns its width toward the camera.

Colors and gradients

`DualRadial()` blends by distance from the local origin of a 2D or 3D shape, including rims. Use `DualRadialBounds()` to fit the gradient to the shape's boundary:

```
MoldStyle centered = MoldStyle.DualRadial(Colors.White, Colors.Blue);
MoldStyle fitted = MoldStyle.DualRadialBounds(Colors.White, Colors.Blue);
```

`DualRadialBounds()` blends from the inner edge to the outer edge on a 3D torus; other 3D primitives treat it as radial. Angular color rotates around local Y from +X toward +Z, fitting a torus's start/span range. Directional gradients also work on 3D shapes. You can choose linear RGB or Oklab interpolation separately from the blend mode. Oklab can give nicer color transitions, but costs extra GPU work. See [blending](#) for the modes and renderer differences.

Dashes

Dashes work on rings, rims, and lines. The count-based pattern below fits 12 periods around the ring, with gaps taking up 45% of each period:

```
MoldDash dash = new(
    fillCount: 12,
    totalCount: 12,
    spacing: 0.45f,
    type: MoldDashType.Chevron,
    modifier: 0.8f);

MoldHandle ring = _mold.Create(
    MoldShape.Ring(1.5f, 0.12f, dash),
    MoldStyle.Additive(Colors.Cyan));

MoldShape dashedLine = MoldShape.Line3D(
    Vector3.Left, Vector3.Right, 0.12f, 0.35f).WithDash(
    new MoldDash(10, 10, 0.5f));
```

For a line, a count-based pattern stretches across the endpoint span. Fixed-length dash patterns are also available when you want the dash and gap lengths to stay in local-space units.

Flat rectangle lines support normal, chevron, and rounded dash profiles. Cylinder lines use the normal profile. Their gaps are shader cuts, so there are no extra cap faces inside those gaps.

Update many shapes at once

Use the bulk APIs when many shapes change together. Reuse the input arrays across frames to avoid creating more temporary data.

```

_mold.UpdatePositions(handles, positions);
_mold.UpdateColors(handles, colors);
_mold.UpdateTransforms(handles, positions, rotations, scales);
_mold.UpdateLinePositions(lineHandles, starts, ends);

```

Here `handles` and `lineHandles` are `MoldHandle[]` arrays, with matching `Vector3[]`, `Quaternion[]`, or `Color[]` values. These methods accept `ReadOnlySpan<T>`, so arrays can be passed directly. If you keep handles in a list, build and reuse an array for bulk animation.

All arrays in a call must have equal lengths. Mold reads them during the call, skips invalid handles, and uses the last value if you include a handle more than once. The return value counts accepted entries, including duplicates; endpoint updates also skip handles that are not lines. Finish any worker writes before passing the arrays to Mold.

Use position-only updates when rotation and scale are unchanged. Color-only updates likewise avoid touching the transform. See [batching](#) for how these edits reach the renderer.

Immediate drawing

For content you want to describe again every frame, the immediate API saves you from keeping a handle for each shape. `BeginImmediate()` replaces the previous immediate frame for that context. `StateScope()` restores your drawing settings when the scope ends:

```

using MoldImmediateFrame draw = _mold.BeginImmediate();
using (draw.StateScope())
{
    draw.Color = Colors.Cyan;
    draw.Thickness = 0.04f;
    draw.Dash = new MoldDash(10, 10, 0.5f);
    draw.Line(Vector3.Left, Vector3.Right);
}

```

Run this drawing code each frame if the content is animated. Commands stay visible until the next immediate frame replaces them or the runtime shuts down. Keep immediate drawing coordinated within a context, since starting another frame replaces what earlier callers drew.

For camera-dependent drawing and editor visualization, derive a tool script from `MoldImmediateDrawer3D`. Mold invokes all drawers in a viewport through one shared frame, so compatible commands still batch together:

```

[Tool]
public partial class AimGuide : MoldImmediateDrawer3D
{
    private float _previewTime;

    [Export(PropertyHint.Range, "0,60,0.1")]
    public float PreviewTime
    {
        get => _previewTime;
        set { _previewTime = value; RequestRedraw(); }
    }

    public override void DrawMolds(MoldImmediateFrame draw, Camera3D? camera)
    {
        draw.Color = Colors.Cyan;
        draw.Line(Vector3.Zero, Vector3.Forward * 2);
    }
}

```

Call `RequestRedraw()` after externally owned preview data changes. Editor cameras automatically trigger another frame when their transform, projection, viewport, or culling mask changes. Editor previews use each shape's requested detail exactly; runtime drawers use the normal screen-space LOD policy. Enable `ContinuouslyRedraw` only for editor animation that changes without an Inspector or transform edit.

Mesh cache and statistics

Prewarm shapes before a busy frame if you want to generate their meshes ahead of time. You can also read the context's statistics to see how many shapes and batches it currently holds:

```
_mold.Prewarm(new[]
{
    MoldShape.Sphere(1.0f, MoldShape.Detail.Medium),
    MoldShape.Cuboid(Vector3.One, 0.35f),
});

MoldStatistics stats = _mold.Statistics;
GD.Print($"{stats.ActiveMoldCount} molds in {stats.BatchCount} batches");
```

ActiveMoldCount counts retained shapes. Use ImmediateCommandCount and ImmediateBatchCount for immediate drawing. BatchCount includes both kinds. Call ClearUnusedMeshes() when you want to drop cached meshes that are no longer in use.

Prewarmed meshes still count toward the unused-cache limit. See [mesh generation](#) for the cache settings.

Managed and SIMD processing

C# defaults to SIMD for cached bounds calculations, using System.Numerics.Vector4 when hardware acceleration is available. You can switch a live context to Managed and check which mode it actually uses:

```
_mold.SetPositionUpdateMode(MoldPositionUpdateMode.Managed);
MoldPositionUpdateMode effective = _mold.EffectivePositionUpdateMode;
bool supported = MoldContext.IsPositionUpdateModeAvailable(
    MoldPositionUpdateMode.SIMD);
```

Set PositionUpdateMode on MoldWorldSettings to choose the initial mode. Settings are copied when the runtime is created, so use SetPositionUpdateMode for later changes.

Unsupported hardware falls back to Managed while keeping the requested mode set to SIMD. Small groups use the scalar calculation in either mode. Switching does not require rebuilding existing bounds. Both modes run synchronously; SIMD speeds up the calculation without moving it to worker threads.

The performance scene has a Managed/SIMD selector. Benchmarks also accept --benchmark-mode=managed or --benchmark-mode=simd after Godot's -- separator. Results include position_update_mode, effective_position_update_mode, and simd_available. Benchmark an exported build on your target hardware to see which works better for your project.

More guides

- [Performance choices](#)
- [Performance test metrics](#)
- [Custom materials](#)
- [Rendering game UI](#)
- [Architecture and runtime ownership](#)

Changelog

0.8.0 — Local NativeCompatible backport (2026-09-11)

- Make C# settings resources serializable through their own named script file; normalize missing/invalid ordering values on reload.

Breaking changes

- Default supported transparent rendering to NativeCompatible; select Batched explicitly for batch-slot rendering. Native mode can increase object, material, and draw overhead.
- Godot C# Batched mode now preserves slot order instead of sorting transparent uploads. Transparent LOD restrictions remain.

Features

- Add NativeCompatible/Batched settings and live world/context switching, pooled native renderers, and native shader instance addressing.
- Add two-mode performance-scene selection with alpha 0.75, selection persistence, 120-frame warmup after switching, and identified benchmark output.
- Preserve existing geometry, immediate APIs, and appearance-buffer layouts. No OIT or later geometry features are included.

This local package is distinct from the existing broader public v0.8.0 release history.

0.7.1 (2026-09-07)

Warning: BREAKING CHANGES

- remove public polyline dash properties, constructor arguments, `WithDash/with_dash`, and authoring controls from both ports; remove these calls and settings when upgrading. Polylines currently render solid strokes. ([eb96f14](#))

Features

- add texture-backed `MoldGpuPolylineRenderer` streams to C# and GDScript for animated planar paths, with point uploads that reuse stroke topology, joins and caps, per-point width and HDR color, manual bounds, and opt-in custom materials. ([ba99bf3](#))
- make Auto select GPU rendering for eligible retained and authored polylines on Forward+, Mobile, and Compatibility, with CPU fallback and stable handles when eligibility changes; editor previews remain CPU-rendered. ([ba99bf3](#))
- add interactive Polyline Backend examples and GPU-polyline regression fixtures for both ports. ([ba99bf3](#))

Documentation

- add GPU-polyline and rendering-backend guides, expand the API and bulk-update documentation, and refresh the project and port READMEs. ([ba99bf3](#))
- build a bookmarked, linked documentation PDF and include it in both release archives. ([894d664](#))

0.7.0 (2026-09-07)

Features

- add camera-aware `MoldImmediateDrawer3D` rendering to both ports, with shared per-viewport batching, editor previews, layer filtering, camera-change redraws, optional continuous redraws, and an editor animation scrubber in the Immediate API examples ([f39e7f6](#))

- add Auto and CPU polyline backend policies to C# and GDScript nodes and context creation APIs; backend resolution now belongs to MoldWorld, with Auto currently selecting the CPU path at the future compute-renderer seam ([9537c03](#))
- expose polyline point colors through Godot's HDR intensity picker in both ports, preserving authored channel values above 1.0 ([9537c03](#))
- support DualRadial, DualRadialBounds, and DualAngular color modes on 3D primitives, including radial-thickness blending for torus bounds ([f85d008](#))

Performance

- reduce immediate recording and synchronization work with bounded preparation caches, independent color and gradient packing, compatible-batch reuse, and transform, bounds, and LOD reuse; add staged CPU benchmarks and mutable-input regression coverage ([f85d008](#))

Bug Fixes

- correct line and polyline subpixel coverage, alpha-to-coverage output, perspective width measurement, endpoint padding, and local-AA fallbacks across retained, immediate, and editor-preview rendering ([d4b94bd](#))

Documentation

- refresh the project and port READMEs for the immediate drawer, expanded color modes, and polyline backend APIs ([e078a43](#), [9537c03](#))

0.6.0 (2026-09-06)

Warning: BREAKING CHANGES

- move the C# Radius and Diameter measurement types from Mold to Mold.MoldMath; add using Mold.MoldMath where these types are referenced ([dc9dfd9](#))
- rename depth-test values from Default/DEFAULT and Inverted/INVERTED to LessEqual/LESS_EQUAL and Greater/GREATER, matching their actual comparisons ([dc9dfd9](#))

Features

- add opt-in renderer, LOD, upload, and payload performance metrics plus richer interactive and JSON benchmark output for both ports ([807e1a5](#), [39c508d](#))
- add bulk color and full-transform updates alongside the existing bulk position and line APIs ([9eeb3d7](#))
- add Managed and SIMD position-processing modes to the C# port, with hardware-aware fallback and per-runtime selection ([ba93042](#))
- accept zero-sized shapes, zero-length lines, and zero-scale transforms as retained but non-drawing molds, allowing them to become visible after later updates ([dc9dfd9](#))

Performance

- reduce transform-update, LOD, bounds, batch-dirtying, and GPU-upload work; cache active slots and avoid repeated unchanged-property writes ([19600c5](#), [9eeb3d7](#), [a09bcfa](#))
- vectorize C# cached-bounds reduction and streamline renderer synchronization in both ports ([ba93042](#))

Bug Fixes

- rework polyline antialiasing with analytic joins, round outlines, endpoint coverage, and robust opaque alpha-to-coverage or fallback handling ([018c393](#))
- correct automatic and explicit depth-write behavior across blend modes, stencil reads, authoring previews, and retained rendering ([dc9dfd9](#))

Documentation

- expand and reorganize the architecture, batching, blending, mesh-generation, performance, custom-material, metrics, and UI-rendering guides ([b66c1bd](#), [39b9186](#))

0.5.1 (2026-09-04)

Bug Fixes

- restore polyline geometry by carrying the screen-space extrusion vector in `ARRAY_CUSTOM0` instead of `ARRAY_TANGENT`; Godot stores tangents octahedron-encoded, which normalized away the extrusion length and inflated every polyline to roughly one world unit per side ([69e8af1](#))
- refresh `mold_viewport_size` on editor previews whenever the viewport resizes, so thin-line antialiasing no longer keeps the size captured at the last property edit ([69e8af1](#))
- publish `mold_viewport_size` to GDScript custom-material previews, matching the C# port and the instanced runtime path ([69e8af1](#))
- guard the polyline edge scale against a zero projected width instead of dividing by an epsilon floor, which could expand a degenerate line by five orders of magnitude ([69e8af1](#))
- restore the canonical `contract/` copies of the shader and mesh-golden fixtures so `tools/sync_contracts.py` no longer reverts the line-rasterization contract in both ports ([69e8af1](#))

0.5.0 (2026-09-03)

Warning: BREAKING CHANGES

- replace the shape-specific rectangle, cuboid, cylinder, and capsule line factories with `Line2D/Line_2d` and `Line3D/Line_3d`; 3D lines now consistently use cylinder topology ([0d989cc](#))
- disable face culling by default; select back- or front-face culling explicitly for one-sided rendering ([6ad8330](#))

Features

- add end caps to partial torus spans and Mold Logo example scenes for both ports ([5d60e17](#))
- add `FaceCameraY/FACE_CAMERA_Y` billboard to keep 2D shapes upright around world Y ([c5ec1de](#))
- add Off, Medium, and High local-antialiasing quality modes and roundness controls for unified 2D and 3D lines ([0d989cc](#))

Improvements

- make thin-line and polyline antialiasing viewport-aware and derivative-based across joins, caps, angular masks, and dashes ([b49b536](#))
- localize rounded 3D fillets by the shortest relevant dimension for `Line3D`, cuboids, cylinders, and regular prisms, preserving straight bodies and planar faces on elongated shapes ([b8fabd3](#), [90623cd](#))

Packaging

- include examples, shared and port documentation, the changelog, and portable documentation links in release archives ([4d85012](#))

0.4.0 (2026-09-02)

Features

- add registered custom `ShaderMaterial` support to the C# and GDScript runtime and authoring APIs, including transparent batch sorting and a documented per-instance shader interface ([bbda52f](#))
- add angular clipping for rectangle and regular-polygon fills and rims, plus corner-local and shape-relative rectangle roundness modes ([0496732](#))

- add C# and GDScript UI showcase scenes and a HUD rendering guide ([7fe178a](#))
- add normal-visualization example scenes for both ports ([b6549b7](#))

Bug Fixes

- calculate automatic-LOD projected diameters from shape-specific bounds ([d567f26](#))

Packaging

- include the proprietary license in generated port archives ([c6859cb](#))

0.3.0 (2026-09-01)

Features

- expand immediate drawing and color styling ([2b22486](#))

If you have questions, feel free contact me at moldsupport@chosenconcept.dev for answers

The below caused some confusions, but here's the TL;DR:

- You're allowed to use Mold in commercial and non-commercial projects.
- You cannot sell, publish, or distribute the source code or anything derived from it.
- You cannot distribute the source code in open source projects.

Not required but I'd be glad to have Mold credited, and/or use #MadeWithMold or tag me on socials:

- X (Twitter): <https://x.com/justdkliao>
- Bluesky: <https://bsky.app/profile/chosenconcept.dev>

Mold Godot Proprietary License

Copyright © 2026 DK Liao (Chosen Concept). All rights reserved.

1. Proprietary rights

Mold Godot, including its source code, shaders, assets, examples, and documentation (the "Software"), is proprietary software owned by DK Liao, publishing under the name Chosen Concept (the "Copyright Holder").

2. No license granted

Unless you have a separate written agreement or written permission from the Copyright Holder, no license or other rights in the Software are granted to you. Without such authorization, you may not use, copy, modify, adapt, translate, merge, publish, distribute, sublicense, sell, rent, lease, make available, create derivative works from, reverse engineer, or otherwise exploit any part of the Software.

Access to the Software through a source-code hosting service does not grant a license beyond the limited rights, if any, required to use that service's features.

3. Third-party materials

This license does not apply to third-party materials identified in `THIRD_PARTY_NOTICES.md` or in the relevant source files. Those materials remain subject to their respective license terms. If a third-party license conflicts with this license for a third-party component, the third-party license controls for that component only.

4. Separate agreements

If the Copyright Holder provides you with a separate written license agreement for the Software, that agreement controls to the extent it conflicts with this license.

5. Disclaimer

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, THE SOFTWARE IS PROVIDED "AS IS" AND WITHOUT WARRANTIES OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE, AND NON-INFRINGEMENT. TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, THE COPYRIGHT HOLDER WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, CONSEQUENTIAL, OR OTHER DAMAGES ARISING FROM OR RELATED TO THE SOFTWARE OR ITS USE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Third-party notices

Portions of the runtime mesh-generation algorithms are adapted from [MetaMesh](#) by Keijiro Takahashi.

MetaMesh is released under the Unlicense:

This is free and unencumbered software released into the public domain.

Anyone is free to copy, modify, publish, use, compile, sell, or distribute this software, either in source code form or as a compiled binary, for any purpose, commercial or non-commercial, and by any means.

In jurisdictions that recognize copyright laws, the author or authors of this software dedicate any and all copyright interest in the software to the public domain. We make this dedication for the benefit of the public at large and to the detriment of our heirs and successors. We intend this dedication to be an overt act of relinquishment in perpetuity of all present and future rights to this software under copyright law.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

For more information, please refer to <https://unlicense.org/>.